

US Patent & Trademark Office

Patent Public Search | Text View

United States Patent Application Publication

20250284458

Kind Code

A1

Publication Date

September 11, 2025

Inventor(s)

NOH; Seock Hwan et al.

MAC ARRAY AND HARDWARE ACCELERATOR INCLUDING THE SAME

Abstract

A hardware accelerator includes a multiply-accumulate (MAC) array, the MAC array including a plurality of arrays, and a global multiplexer logic configured to connect or disconnect at least two of the plurality of arrays, based on a first control signal. Each of the plurality of arrays includes a plurality of sub-arrays including a plurality of first sub-arrays and a second sub-array, and a local multiplexer logic configured to connect or disconnect the plurality of sub-arrays, based on a second control signal.

Inventors: NOH; Seock Hwan (Incheon, KR), KUNG; Jae Ha (Seoul, KR), KOO; Ja Hyun (Daegu, KR), LEE; Seung Pyo (Daegu, KR)

Applicant: DAEGU GYEONGBUK INSTITUTE OF SCIENCE AND TECHNOLOGY (Daegu, KR); KOREA UNIVERSITY RESEARCH AND BUSINESS FOUNDATION (Seoul, KR)

Family ID: 1000008035349

Assignee: DAEGU GYEONGBUK INSTITUTE OF SCIENCE AND TECHNOLOGY (Daegu, KR); KOREA UNIVERSITY RESEARCH AND BUSINESS FOUNDATION (Seoul, KR)

Appl. No.: 18/763042

Filed: July 03, 2024

Foreign Application Priority Data

KR

10-2024-0031559

Mar. 05, 2024

Publication Classification

Int. Cl.: G06F7/544 (20060101); G06F7/499 (20060101)

Background/Summary

CROSS-REFERENCE TO RELATED APPLICATION

[0001] This application is based on and claims priority under 35 U.S.C. § 119 to Korean Patent Application No. 10-2024-0031559, filed on Mar. 5, 2024, in the Korean Intellectual Property Office, the disclosure of which is incorporated by reference herein in its entirety.

BACKGROUND

1. Field

[0002] The disclosure relates to a multiply-accumulate (MAC) array and a hardware accelerator including the same, and more particularly, to a method of mapping data to a MAC operator that performs operations of a deep neural network model, an arrangement structure of MAC operators included in a MAC array, and a method of transmitting and receiving data between the MAC operators.

[0003] The disclosure relates to a multiplier that supports various precisions and datatypes and an operating method thereof, and more particularly, to a multiplier that identifies precisions and datatypes of values input in units of bits and adaptively performs a multiplication operation and an operating method of the multiplier.

[0004] This research has been conducted with support from the Samsung Future Technology Promotion Project (Project Number: SRFC-IT1902-03).

2. Description of the Related Art

[0005] High-performance computing systems and continuously growing open-source datasets have led to extremely quick advances of artificial intelligence technology. In addition, along with the improved accuracy in artificial intelligence technology, artificial intelligence technology is used in many application fields, such as computer vision, language modeling, and autonomous driving.

[0006] To use artificial intelligence for applications, training processes are required. In artificial intelligence technology, training refers to a process of updating weights of artificial intelligence models (for example, deep neural networks (DNNs)) by using specific datasets. As the weights are better updated, artificial intelligence models may perform given tasks better.

[0007] However, because training processes require extremely large amounts of calculation, training via central processing units (CPUs) takes extremely much time. Although graphics processing units (GPUs) facilitate parallel processing and thus less training time is required than in the case of CPUs, GPUs exhibit low usage due to the structural nature thereof. Recently, to overcome the drawbacks of CPUs and GPUs, a lot of dedicated hardware accelerators for performing calculations in DNNs have been proposed.

[0008] In addition, as types of DNN models become more diverse, and multiple users request operations of DNN models in a multitenant environment, the demand for parallel processing functions of dedicated hardware accelerators is increasing.

[0009] Meanwhile, as datatypes and precisions of values required for operations of DNN models become more diverse, and multiply-accumulate (MAC) operators support various datatypes and precisions, the demand for area-efficient hardware accelerators has increased.

SUMMARY

[0010] Additional aspects will be set forth in part in the description which follows and, in part, will be apparent from the description, or may be learned by practice of the presented embodiments of the disclosure.

[0011] According to an aspect of the disclosure, a hardware accelerator includes a multiply-accumulate (MAC) array. The MAC array includes a plurality of arrays, and a global multiplexer logic configured to connect or disconnect at least two of the plurality of arrays, based on a first control signal. Each of the plurality of arrays includes a plurality of sub-arrays including a plurality of first sub-arrays and a second sub-array, and a local multiplexer logic configured to connect or disconnect the plurality of sub-arrays, based on a second control signal.

[0012] The MAC array may include four arrays, each including seven first sub-arrays, one second sub-array, and a local multiplexer logic, and a global multiplexer logic disposed between the arrays. Each of the first sub-arrays may include a plurality of MAC operators arranged in the form of 9×64 , the second sub-array may include a plurality of MAC operators arranged in the form of 1×64 , and the local multiplexer logic may be disposed between the first sub-arrays and the second sub-array.

Description

BRIEF DESCRIPTION OF THE DRAWINGS

[0013] This disclosure may be readily understood by reference to the following detailed description and the accompanying drawings, in which reference numerals refer to structural elements.

[0014] The above and other aspects, features, and advantages of certain embodiments of the inventive concept will be more apparent from the following description taken in conjunction with the accompanying drawings, in which:

[0015] FIG. 1 is a block diagram for explaining the configuration of a hardware accelerator according to an embodiment of the disclosure;

[0016] FIG. 2 is a block diagram for explaining the configuration of a multiply-accumulate (MAC) array according to an embodiment of the disclosure;

[0017] FIG. 3 is a block diagram for explaining the configuration of a first array according to an embodiment of the disclosure;

[0018] FIG. 4 is a block diagram for explaining the operation of a MAC array according to an embodiment of the disclosure;

[0019] FIG. 5 is a conceptual diagram for explaining convolution operations according to an embodiment of the disclosure;

[0020] FIG. 6 is a conceptual diagram illustrating the operation of a MAC array that performs an accumulated convolution operation according to an embodiment of the disclosure;

[0021] FIG. 7 is a conceptual diagram illustrating the operation of a first sub-array that performs a non-accumulated convolution operation according to an embodiment of the disclosure;

[0022] FIG. 8 is a conceptual diagram for explaining the configuration and operation of a second sub-array according to an embodiment of the disclosure;

[0023] FIG. 9 is a block diagram for explaining the configuration of a MAC operator according to an embodiment of the disclosure;

[0024] FIG. 10 is a block diagram for explaining the configuration of a multiplier according to an embodiment of the disclosure;

[0025] FIGS. 11A to 11C are conceptual diagrams for explaining operations of a multiplier according to an embodiment of the disclosure;

[0026] FIG. 12 is a block diagram for explaining the configuration of multiplication logic according to an embodiment of the disclosure;

[0027] FIG. 13 is a conceptual diagram illustrating a multiplication operation according to an embodiment of the disclosure;

[0028] FIGS. 14A and 14B are block diagrams for explaining operations of a multiplication operation according to an embodiment of the disclosure;

[0029] FIG. **15** is a block diagram for explaining the configuration of addition logic according to an embodiment of the disclosure;

[0030] FIG. **16** is a block diagram for explaining the configuration of sub-addition logic according to an embodiment of the disclosure;

[0031] FIG. **17** is a block diagram for explaining the configuration of XOR logic according to an embodiment of the disclosure;

[0032] FIG. **18** is a block diagram for explaining the configuration of an electronic device according to an embodiment of the disclosure;

[0033] FIG. **19** is a flowchart for explaining the operation of a multiplier according to an embodiment of the disclosure;

[0034] FIG. **20** is a flowchart for explaining the operation of shifting logic according to an embodiment of the disclosure;

[0035] FIG. **21** is a flowchart for explaining the operation of a selective adder according to an embodiment of the disclosure;

[0036] FIG. **22** is a flowchart for explaining the operation of addition logic according to an embodiment of the disclosure;

[0037] FIG. **23** is a flowchart for explaining the operation of XOR logic according to an embodiment of the disclosure; and

[0038] FIG. **24** is a flowchart for explaining the operations of a normalizer and a rounder according to an embodiment of the disclosure.

DETAILED DESCRIPTION

[0039] Throughout the disclosure, the expression “at least one of a, b or c” indicates only a, only b, only c, both a and b, both a and c, both b and c, all of a, b, and c, or variations thereof.

[0040] Although general terms widely used at present were selected for describing the disclosure in consideration of the functions thereof, these general terms may vary according to intentions of one of ordinary skill in the art, case precedents, the advent of new technologies, or the like. Terms arbitrarily selected by the applicant of the disclosure may also be used in a specific case. In this case, their meanings need to be given in the detailed description of an embodiment of the disclosure. Hence, the terms must be defined based on their meanings and the contents of the entire specification, not by simply stating the terms.

[0041] An expression used in the singular may encompass the expression of the plural, unless it has a clearly different meaning in the context. Unless otherwise defined, all terms (including technical and scientific terms) used herein have the same meaning as commonly understood by one of ordinary skill in the art to which this disclosure belongs.

[0042] The terms “comprises” and/or “comprising” or “includes” and/or “including” when used in this specification, specify the presence of stated elements, but do not preclude the presence or addition of one or more other elements. The terms “unit”, “-er (-or)”, and “module” when used in this specification refers to a unit in which at least one function or operation is performed, and may be implemented as hardware, software, or a combination of hardware and software.

[0043] The expression “configured to (or set to)” used therein may be used interchangeably with, for example, “suitable for”, “having the capacity to”, “designed to”, “adapted to”, “made to”, or “capable of”, according to situations. The expression “configured to (or set to)” may not only necessarily refer to “specifically designed to” in terms of hardware. Instead, in some situations, the expression “system configured to” may refer to a situation in which the system is “capable of” together with another device or parts. For example, the phrase “a processor configured (or set) to perform A, B, and C” may mean a dedicated processor (such as an embedded processor) for performing a corresponding operation, or a generic-purpose processor (such as a central processing unit (CPU) or an application processor (AP)) that can perform a corresponding operation by executing one or more software programs stored in a memory.

[0044] When an element (e.g., a first element) is “coupled to” or “connected to” another element

(e.g., a second element), the first element may be directly coupled to or connected to the second element, or, unless otherwise described, a third element may exist therebetween.

[0045] In the disclosure, functions related to artificial intelligence (AI) are operated through a processor and a memory. The processor may include one or a plurality of processors. The one or plurality of processors may be a general-purpose processor such as a central processing unit (CPU), an application processor (AP), or a digital signal processor (DSP), a graphics-only processor such as a graphics processing unit (GPU) or a vision processing unit (VPU), or an AI-only processor such as a neural processing unit (NPU). The one or plurality of processors control to process input data, according to a predefined operation rule or AI model stored in the memory. Alternatively, when the one or plurality of processors are AI-only processors, the AI-only processors may be designed in a hardware structure specialized for processing a specific AI model.

[0046] The predefined operation rule or AI model is characterized in that it is created through learning. Here, being created through learning means that a basic AI model is trained using a plurality of training data by a learning algorithm, so that a predefined operation rule or AI model set to perform desired characteristics (or a desired purpose) is created. Such learning may be performed in a device itself on which AI according to the disclosure is performed, or may be performed through a separate server and/or system. Examples of the learning algorithm include, but are not limited to, supervised learning, unsupervised learning, semi-supervised learning, or reinforcement learning.

[0047] According to an embodiment of the disclosure, an 'AI model' may include a neural network model. The AI model may be composed of a plurality of neural network layers. Each of the plurality of neural network layers has a plurality of weight values, and performs a neural network operation through an operation between an operation result of a previous layer and the plurality of weight values. The plurality of weight values of the plurality of neural network layers may be optimized by a learning result of the AI model. For example, the plurality of weight values may be updated so that a loss value or a cost value obtained from the AI model is reduced or minimized during a learning process. The artificial neural network may include a deep neural network (DNN), for example, a Convolutional Neural Network (CNN), a Recurrent Neural Network (RNN), a Restricted Boltzmann Machine (RBM), a Deep Belief Network (DBN), a Bidirectional Recurrent Deep Neural Network (BRDNN), or a Deep Q-Networks, but embodiments of the disclosure are not limited thereto.

[0048] In the disclosure, a tensor may denote an n-dimensional array of data. Each dimension of the tensor may form an axis. For example, a 0-dimensional tensor may represent a scalar value, a 1-dimensional tensor may represent a vector value, and a 2-dimensional tensor may represent a matrix. For example, a 3 or more-dimensional tensor may include a plurality of matrices and may form 3 or more axes.

[0049] In the disclosure, a batch may indicate a unit for updating parameters by grouping all datasets for the training of the artificial intelligence model. For example, all the datasets may be grouped into a plurality of batches. The parameters of the artificial intelligence model may be updated for every one batch. One batch may include a predefined number of mini-batches.

[0050] In the disclosure, multiply-accumulate (MAC) operators may be arranged in an $n \times m$ array in the form of n rows and m columns. The $n \times m$ array may perform $n \times m$ matrix operations by using the MAC operators.

[0051] In the disclosure, the expression "the processor may perform an A operation" may indicate that the processor may execute at least one instruction corresponding to the A operation.

[0052] Embodiments of the disclosure are described in detail herein with reference to the accompanying drawings so that this disclosure may be easily performed by one of ordinary skill in the art to which the disclosure pertains. The disclosure may, however, be embodied in many different forms and should not be construed as being limited to the examples set forth herein.

[0053] FIG. 1 is a block diagram for explaining the configuration of a hardware accelerator **1000**

according to an embodiment of the disclosure.

[0054] Referring to FIG. 1, the hardware accelerator **1000** may include a multiply-accumulate (MAC) array **1100**, a scratchpad memory **1200**, a direct memory access (DMA) controller **1300**, a central controller **1400**, and high bandwidth memory (HBM) **1500**. However, not all of the illustrated components are essential. The hardware accelerator **1000** may be implemented by more or less components than those illustrated in FIG. 1.

[0055] The MAC array **1100** may include a plurality of MAC operators. In the disclosure, a MAC operator may also be referred to as a MAC unit. The MAC operator may perform multiplication between input values. The MAC operator (e.g., a first MAC operator) may receive an output (or may also be referred to as a partial sum output) of an adjacent MAC operator (e.g., a second MAC operator). The MAC operator (e.g., the first MAC operator) may sum the output of the adjacent MAC operator (e.g., the second MAC operator) and a multiplication result. The MAC operator (e.g., the first MAC operator) may transmit a summation result to another adjacent MAC operator (e.g., a third MAC operator).

[0056] The MAC array **1100** may perform operations of at least one deep neural network (DNN) model. The operations of the DNN model may be required for learning or inference of the DNN model. For example, the operations of the DNN model may be classified into DNN operations and non-DNN operations. DNN operations may include non-cumulative operations such as weight gradient operation, depthwise (DW) convolution, dilated convolution, and up convolution, and cumulative operations such as (general) convolution, pointwise convolution, or fully connected layer operation. Non-DNN operations may include operations in non-DNN layers, such as ReLU, batch normalization, and softmax functions in the DNN model.

[0057] In an embodiment of the disclosure, the MAC array **1100** may perform DNN operations by using a plurality of MAC operators. The MAC array **1100** may group the plurality of MAC operators into at least one group. The at least one group may perform different DNN operations. In an embodiment of the disclosure, the MAC operators may be arranged in the MAC array **1100** in the form of 128×128. The specific configuration and operation of the MAC array **1100** will be described in detail with reference to FIG. 2.

[0058] In an embodiment of the disclosure, unlike shown in FIG. 1, the hardware accelerator **1000** may include a vector unit. The vector unit may include a plurality of arithmetic and logic units (ALUs). The vector unit may perform non-DNN operations by using the plurality of ALUs.

[0059] According to an embodiment of the disclosure, the MAC array **1100** may perform an operation on a plurality of DNN models or perform an operation on one DNN model in a multitenant environment, by combining the plurality of MAC operators into the at least one group or dividing the at least one group. According to an embodiment of the disclosure, the MAC array **1100** may support the multitenant environment through a flexible structure, thereby increasing hardware resource utilization with respect to the plurality of MAC operators.

[0060] The scratchpad memory **1200** may store data corresponding to operands required for the operation of the DNN model. For example, the scratchpad memory **1200** may store input tensors and/or weight tensors. The scratchpad memory **1200** may perform a function of prefetching data from an external memory **10** through the DMA controller **1300**.

[0061] The MAC array **1100** and/or the vector unit may fetch data from the scratchpad memory **1200**. The scratchpad memory **1200** may perform a function of prefetching data from the external memory **10** through the DMA controller **1300**.

[0062] The MAC array **1100** and/or the vector unit may store data in the scratchpad memory **1200**. The scratchpad memory **1200** may store partial sum outputs calculated from the MAC array **1100** and/or the vector unit.

[0063] According to an embodiment of the disclosure, a compiler of the scratchpad memory **1200** may easily predict a memory access pattern by a deterministic data flow of the DNN operation. However, embodiments of the disclosure are not limited thereto. In an embodiment of the

disclosure, the scratchpad memory **1200** may be omitted, and the hardware accelerator **1000** may include a static random access memory (SRAM) cache.

[0064] The DMA controller **1300** may control data transmission between internal memories (e.g., the scratchpad memory **1200** and/or the HBM **1500**), data transmission between an internal memory and the external memory **10**, and/or data transmission between an internal memory and the MAC array **1100**. The hardware accelerator **1000** may load data and/or programs from the external memory **10** by using the DMA controller **1300**. The hardware accelerator **1000** may transmit data to the external memory **10** by using the DMA controller **1300**.

[0065] The central controller **1400** may control the overall operations of the hardware accelerator **1000**. The central controller **1400** may transmit control signals to the MAC array **1100**, the scratchpad memory **1200**, and the DMA controller **1300**. The central controller **1400** may store program codes or instructions for controlling the hardware accelerator **1000**. The central controller **1400** may support instructions such as interrupt, halt, synchronization, and no-operation (NOP). In an embodiment of the disclosure, the central controller **1400** may be implemented as a RISC-V CPU.

[0066] The central controller **1400** may transmit, to the MAC array **1100**, a control signal for combining the MAC operators of the MAC array **1100** into the at least one group or dividing the at least one group. The central controller **1400** may transmit, to the MAC array **1100**, a control signal for activating and/or deactivating at least one operation logic included in a multiplier of the MAC operator.

[0067] The HBM **1500** may store data required for the operation of the DNN model. The capacity of the HBM **1500** may be greater than the capacity of the scratchpad memory **1200**. The HBM **1500** may receive data from the scratchpad memory **1200** or transmit data to the scratchpad memory **1200**, through control of the DMA controller **1300**.

[0068] FIG. **1** shows that the hardware accelerator **1000** includes the scratchpad memory **1200** and the HBM **1500** as internal memories, but the disclosure is not limited thereto. For example, the hardware accelerator **1000** may include, as internal memories, at least one type of storage medium selected from among a flash memory type, a hard disk type, a multimedia card micro type, a card type memory (for example, a secure digital (SD) or extreme digital (XD) memory), a random access memory (RAM), a static random access memory (SRAM), a read-only memory (ROM), an electrically erasable programmable ROM (EEPROM), a programmable ROM (PROM), magnetic memory, a magnetic disk, and an optical disk.

[0069] FIG. **2** is a block diagram for explaining the configuration of the MAC array according to an embodiment of the disclosure. A description of FIG. **2** that is the same as given above with reference to FIG. **1** will not be repeated herein. For convenience of description, it is described that the MAC array **1100** is a MAC array in which 128×128 MAC operators are arranged, but the disclosure is not limited thereto. For example, the MAC array **1100** may be a MAC operator in which $m \times n$ MAC operators are arranged. Here, m and n may be natural numbers.

[0070] Referring to FIG. **1** along with FIG. **2**, the MAC array **1100** may include a plurality of arrays **1110** and global multiplexer logic **1120**. The plurality of arrays **1110** may include the same numbers of MAC operators. For convenience of description, it is described that each of the plurality of arrays **1110** is a MAC array in which 64×64 MAC operators are arranged, but the disclosure is not limited thereto.

[0071] The MAC array **1100** may include four 64×64 arrays, namely, first, second, third, and fourth arrays **1110_1**, **1110_2**, **1110_3**, and **1110_4**. However, the disclosure is not limited thereto, and the MAC array **1100** may be configured to include smaller and more arrays, or include larger and fewer arrays.

[0072] The global multiplexer logic **1120** may connect or disconnect at least two of the plurality of arrays **1110**. The global multiplexer logic **1120** may receive a first control signal from the central controller **1400**. The first control signal may correspond to whether the plurality of arrays **1110** are

connected to each other.

[0073] In an embodiment of the disclosure, the first control signal may correspond to an operation mode of the MAC array **1100**. For example, in a first operation mode, the global multiplexer logic **1120** may disconnect the plurality of arrays **1110** from each other. For example, in a second operation mode, the global multiplexer logic **1120** may connect at least two of the plurality of arrays **1110** to each other.

[0074] The connected arrays **1110** may transmit and receive data to and from each other. For example, when the first array **1110_1** and the second array **1110_2** are connected to each other, the first array **1110_1** and the second array **1110_2** may perform a 128×64 matrix operation. For example, when the first array **1110_1** and the third array **1110_3** are connected to each other, the first array **1110_1** and the third array **1110_3** may perform a 64×128 matrix operation. For example, when the first array **1110_1**, the second array **1110_2**, the third array **1110_3**, and the fourth array **1110_4** are all connected to each other, the first array **1110_1**, the second array **1110_2**, the third array **1110_3**, and the fourth array **1110_4** may perform a 128×128 matrix operation.

[0075] For example, among the plurality of arrays **1110**, two arrays (e.g., the second array **1110_2** and the fourth array **1110_4**) may be grouped, and the remaining arrays may not be grouped. The two grouped arrays may perform a 128×64 matrix operation or a 64×128 matrix operation. The two grouped arrays may process an operation of a DNN model with a preset higher priority, and the two ungrouped arrays may process an operation of a DNN model with a preset lower priority.

[0076] The global multiplexer logic **1120** may include a plurality of multiplexers. Each of the plurality of multiplexers may or may not transfer an output of one array (e.g., the first array **1110_1**) among the plurality of arrays **1110** to an adjacent array (e.g., the third array **1110_3**) in response to the first control signal. For example, each of the plurality of multiplexers may transfer an output of one array (e.g., the first array **1110_1**) among the plurality of arrays **1110** to an adjacent array (e.g., the third array **1110_3**), based on the first control signal corresponding to a first logic value. For example, each of the plurality of multiplexers may not transfer an output of one array (e.g., the first array **1110_1**) among the plurality of arrays **1110** to an adjacent array (e.g., the third array **1110_3**), based on the first control signal corresponding to a first logic value.

[0077] In an embodiment of the disclosure, the plurality of arrays **1110** may be grouped into at least one array group by the global multiplexer logic **1120**. The MAC array **1100** may perform operations on a plurality of tenants. Each of the at least one array group may perform operations on different single tenants among the plurality of tenants.

[0078] The plurality of multiplexers may be connected between a plurality of MAC operators disposed in a last row, a last column, a first row, or a first column of the arrays.

[0079] In an embodiment of the disclosure, the MAC operators arranged in a last row of the first array **1110_1** may be respectively connected to the plurality of multiplexers. The corresponding multiplexers may be respectively connected to the MAC operators arranged in a first row of the third array **1110_3**.

[0080] In an embodiment of the disclosure, the MAC operators arranged in a last column of the first array **1110_1** may be respectively connected to the plurality of multiplexers. The corresponding multiplexers may be respectively connected to the MAC operators arranged in a first column of the second array **1110_2**.

[0081] In an embodiment of the disclosure, the MAC operators arranged in a last column of the third array **1110_3** may be respectively connected to the plurality of multiplexers. The corresponding multiplexers may be respectively connected to the MAC operators arranged in a first column of the fourth array **1110_4**.

[0082] In an embodiment of the disclosure, the MAC operators arranged in a last row of the second array **1110_2** may be respectively connected to the plurality of multiplexers. The corresponding multiplexers may be respectively connected to the MAC operators arranged in a first row of the fourth array **1110_4**.

[0083] FIG. 3 is a block diagram for explaining the configuration of the first array **1110_1** according to an embodiment of the disclosure. Matters of FIG. 3 that are the same as those described above with reference to FIGS. 1 and 2 will not be repeated herein.

[0084] Referring to FIGS. 1 and 2 along with FIG. 3, the first array **1110_1** may include a plurality of sub-arrays including a plurality of first sub-arrays **1111** and a second sub-array **1112**. The first array **1110_1** may include local multiplexer logic **1113**.

[0085] In an embodiment of the disclosure, MAC operators may be arranged in each of the plurality of first sub-arrays **1111** in the form of 9×64 . In an embodiment of the disclosure, the number of first sub-arrays **1111** may be 7. For example, the first array **1110_1** may include seven first sub-arrays **1111_1** to **1111_7**.

[0086] In an embodiment of the disclosure, MAC operators may be arranged in the second sub-array **1112** in the form of 1×64 . The specific structure of the second sub-array **1112** will be described in detail in FIG. 8.

[0087] The local multiplexer logic **1113** may be disposed between the plurality of sub-arrays. For example, local multiplexer logics **1113_1** to **1113_7** may be disposed between the first sub-arrays **1111_1** to **1111_7** and the second sub-array **1112**. The local multiplexer logic **1113** may connect or disconnect the plurality of sub-arrays. For example, the local multiplexer logic **1113_1** may be disposed between the first sub-array **1111_1** and the first sub-array **1111_2**. The local multiplexer logic **1113_1** may connect or disconnect the first sub-array **1111_1** and the first sub-array **1111_2**. For example, the local multiplexer logic **1113_7** may be disposed between the first sub-array **1111_7** and the second sub-array **1112**. The local multiplexer logic **1113_7** may connect or disconnect the first sub-array **1111_7** and the second sub-array **1112** to or from each other.

[0088] In an embodiment of the disclosure, the local multiplexer logic **1113** may connect or disconnect the plurality of sub-arrays to or from each other based on a second control signal. The local multiplexer logic **1113** may receive the second control signal from the central controller **1400**. The second control signal may correspond to whether the plurality of sub-arrays are connected to each other.

[0089] In an embodiment of the disclosure, the second control signal may correspond to an operation mode of the first array **1110_1**. For example, in a first operation mode, the local multiplexer logic **1113** may disconnect the plurality of sub-arrays from each other. For example, in a second operation mode, the local multiplexer logic **1113** may connect the plurality of sub-arrays to each other.

[0090] In an embodiment of the disclosure, the plurality of sub-arrays may perform a convolution operation between a first tensor and a second tensor. The plurality of sub-arrays may generate a partial sum tensor as a result of a convolution operation. For example, based on the second control signal indicating the first operation mode of the first array **1110_1**, the first array **1110_1** may accumulate partial sum tensors respectively output by the plurality of sub-arrays. For example, based on the second control signal indicating the second operation mode of the first array **1110_1**, the first array **1110_1** may not accumulate partial sum tensors respectively output by the plurality of sub-arrays.

[0091] In an embodiment of the disclosure, it will be described below under the assumption that the first array **1110_1** operates in the first operation mode. The first array **1110_1** may perform a 64×64 matrix operation. For example, an output of the first sub-array **1111_1** may be transferred to the first sub-array **1111_2** through the local multiplexer logic **1113_1**. An output of the first sub-array **1111_2** may be transferred to the first sub-array **1111_3** through the local multiplexer logic **1113_2**. An output of the first sub-array **1111_3** may be transferred to the first sub-array **1111_4** through the local multiplexer logic **1113_3**. An output of the first sub-array **1111_4** may be transferred to the first sub-array **1111_5** through the local multiplexer logic **1113_4**. An output of the first sub-array **1111_5** may be transferred to the first sub-array **1111_6** through the local multiplexer logic **1113_5**. An output of the first sub-array **1111_6** may be transferred to the first sub-array **1111_7** through the

local multiplexer logic **1113_6**. An output of the first sub-array **1111_7** may be transferred to the second sub-array **1112_7** through the local multiplexer logic **1113_7**.

[0092] In an embodiment of the disclosure, it will be described below under the assumption that the first array **1110_1** operates in the second operation mode. Each of the outputs of the first sub-arrays **1111_1** to **1111_7** and the second sub-array **1112** may be transferred to the scratchpad memory **1200**.

[0093] For convenience of description, the first array **1110_1** has been described as an example, but the configurations, functions, and operations of the plurality of arrays **1100** (e.g., the second array **1110_2**, the third array **1110_3**, and the fourth array **1110_4** in FIG. 2) included in the MAC array **1100** of FIG. 2 may respectively correspond to the configuration, function, and operation of the first array **1110_1**.

[0094] FIG. 4 is a block diagram for explaining the operation of the MAC array **1100** according to an embodiment of the disclosure. Matters of FIG. 4 that are the same as those described above with reference to FIGS. 1 to 3 will not be repeated herein. For convenience of description, at least some of the components shown in FIGS. 1 to 3 are omitted in FIG. 4.

[0095] Referring to FIGS. 1 to 3 along with FIG. 4, the MAC array **1100** may include a plurality of MAC operators. The MAC operator may transfer a partial sum output to another MAC operator (e.g., a lower MAC operator) in a column direction. The MAC operator may forward data of an input tensor to another MAC operator (e.g., a right MAC operator) in a row direction.

[0096] The MAC array **1100** may include the first array **1110_1**, the second array **1110_2**, and the global multiplexer logic **1120**. Multiplexers of the global multiplexer logic **1120** may be disposed between the MAC operators in the last row of the first array **1110_1** and the MAC operators in the first row of the second array **1110_2**. For example, outputs (e.g., forwarded data) of the MAC operators in the last row of the first array **1110_1** may be respectively transferred to the multiplexers of the global multiplexer logic **1120**.

[0097] Each of the MAC operators in the last row of the first array **1110_1** may correspond to one multiplexer. By an operation of one multiplexer that operates based on a logic value of a first control signal, the output of each of the MAC operators in the last row of the first array **1110_1** or data from the scratchpad memory **1200** may be transferred to each of the MAC operators in the first column of the second array **1110_2**. For example, when the first control signal has the first logical value (e.g., '0'), the respective outputs (e.g., forwarded data) of the MAC operators in the last row of the first array **1110_1** may be respectively transferred to the MAC operators in the first column of the second array **1110_2**. For example, when the first control signal has a second logic value (e.g., '1'), the data from the scratchpad memory **1200** may be transferred to each of the MAC operators in the first column of the second array **1110_2**.

[0098] The first array **1110_1** may include the first sub-array **1111_1**, the first sub-array **1111_2**, and the local multiplexer logic **1113_1**. MAC operators in a last row of the first sub-array **1111_1** may be respectively connected to multiplexers of the local multiplexer logic **1113_1**. Respective outputs (e.g., partial sum outputs) of the MAC operators in the last row of the first sub-array **1111_1** may be respectively connected to the multiplexers of the local multiplexer logic **1113_1**.

[0099] Each of the MAC operators in the last row of the first sub-array **1111_1** may correspond to two multiplexers. By operations of the two multiplexers operating based on a logic value of the second control signal, the output of each of the MAC operators in the last row may be transferred to the scratchpad memory **1200** or to each of MAC operators in a first row of the first sub-array **1111_2**. For example, when the second control signal has a first logic value (e.g., '0'), the output of the MAC operator may be transferred to the scratchpad memory **1200**. For example, when the second control signal has a second logic value (e.g., '1'), the output of the MAC operator may be transferred to the first sub-array **1111_2**.

[0100] The second array **1110_2** may include a first sub-array **1111_8** and a first sub-array **1111_9**. The configurations, operations, and functions of the first sub-array **1111_8**, the first sub-array

1111_9, and the local multiplexer logic **1113_8** may respectively correspond to the configurations, operations, and functions of the first sub-array **1111_1**, the first sub-array **1111_2**, and the local multiplexer logic **1113_1**.

[0101] FIG. 5 is a conceptual diagram for explaining convolution operations according to an embodiment of the disclosure.

[0102] Referring to FIG. 1 along with FIG. 5, the MAC array **1100** may perform convolution operations between an input tensor and a weight tensor. According to types of convolution operations, the MAC array **1100** may obtain a first output that accumulates partial sum tensors that are results of a convolution operation between the input tensor and the weight tensor, or may obtain a second output that does not accumulate the partial sum tensors.

[0103] The convolution operations may be classified into a first operation that accumulates partial sum tensors in a direction of an input channel I.C of the input tensor, and a second operation that does not accumulate partial sum tensors in the direction of the input channel I.C of the input tensor. Examples of operations are shown in Table 1 below.

TABLE-US-00001 TABLE 1 Types of operations Examples Accumulation? Yes Conventional convolution, Fully-connected layer, GEMM, GEMV Accumulation? No Depthwise convolution, Dilated convolution, Weight gradient ($\partial \text{custom-character} / \partial W$) in the training process of convolutions

[0104] Referring to Table 1, for example, when a weight gradient operation, a DW convolution, a dilated convolution, etc. in training of a DNN model is performed, the MAC array **1100** may perform a first operation. For example, when performing a (general) convolution in which operation results are accumulated, a fully connected layer operation, a general matrix-matrix multiplication (GEMM) operation, a general matrix vector multiplication (GEMV) operation, etc., the MAC array **1100** may perform the first operation.

[0105] In an embodiment of the disclosure, the MAC array **1100** may identify which operation is required at each step of an operation of at least one DNN model currently being operated. The MAC array **1100** may selectively accumulate and output operation results according to results of the identification.

[0106] FIG. 6 is a conceptual diagram illustrating the operation of a MAC array that performs an accumulated convolution operation according to an embodiment of the disclosure. Matters of FIG. 6 that are the same as those described above with reference to FIGS. 1 to 5 will not be repeated herein.

[0107] Referring to FIG. 6, input training data may include a plurality of batches. Each of the plurality of batches may include an input tensor, which is a three-dimensional (3D) tensor with a depth of the input channel I.C. Data of input tensors corresponding to the plurality of batches may be prefetched to the MAC array in the form of “dataflow algorithm of oblique input streaming”.

[0108] When the accumulated convolution operation is performed in a direction of the input channel I.C of the input tensor, data in the direction of the input channel I.C of the input tensor may be mapped in a column direction of the MAC operators of the MAC array.

[0109] Weight values that perform the convolution operation on input training data may include weight tensors, which are 3D tensors with a depth of the input channel I.C. The number of weight tensors may be equal to the number of output channels O.C. When the accumulated convolution operation is performed in the direction of the input channel I.C of the input tensor, data in a direction of the output channel O.C of the weight tensor may be mapped in a row direction of the MAC operators of the MAC array.

[0110] FIG. 6 shows a 4×4 array, but referring to FIG. 2 along with FIG. 6, data of the input channel I.C. may be input in the column direction of the plurality of arrays **1110** in the form of 64×64, and data of the output channel O.C may be input in the row direction.

[0111] FIG. 7 is a conceptual diagram illustrating the operation of a sub-array that performs a non-accumulated convolution operation according to an embodiment of the disclosure. Matters of FIG.

7 that are the same as those described above with reference to FIGS. 1 to 6 will not be repeated herein.

[0112] Referring to FIG. 7, MAC operators may be disposed in the sub-array (e.g., the first sub-array **1111_1** in FIG. 3) in the form of 9×64 . When the non-accumulated convolution operation is performed in a direction of the input channel I.C of an input tensor, elements of a weight filter may be mapped in a column direction of the sub-array, and data in a direction of the output channel O.C may be mapped in a row direction. The elements of the weight filter may be mapped in the column direction of the sub-array, and the data in the direction of the output channel O.C may be prefetched in the row direction. For example, the weight filter may correspond to one channel of a weight tensor, which is a 3D tensor.

[0113] Similar to that described above with reference to FIG. 6, training data may include input tensors corresponding to a plurality of batches. The input tensors may be input to the sub-array in the form of “dataflow algorithm of oblique input streaming”.

[0114] Because partial sums are not accumulated in the direction of the input channel I.C, a final output from one column of the sub-array may not be transferred to a MAC operator of another sub-array, but transferred to a memory (e.g., the scratchpad memory **1200** in FIG. 1).

[0115] FIG. 8 is a conceptual diagram for explaining the configuration and operation of the second sub-array **1112** according to an embodiment of the disclosure. Matters of FIG. 8 that are the same as those described above with reference to FIGS. 1 to 7 will not be repeated herein.

[0116] Referring to FIG. 3 along with FIG. 8, the second sub-array **1112** may include 64 MAC operators **MAC0** to **MAC63**.

[0117] When an accumulated operation is performed in a direction of an input channel, the second sub-array **1112** may not be internally divided, and may perform a 64×64 matrix operation together with the first sub-arrays **1111**.

[0118] When a non-accumulated operation is performed in the direction of the input channel, the second sub-array **1112** may be divided into a plurality of groups. For example, the second sub-array **1112** may be divided into seven groups. The second sub-array **1112** may be divided into first to seventh groups **1112a** to **1112g**. Each of the first to seventh groups **1112a** to **1112g** may perform 9 MAC operations and output results of the 9 MAC operations.

[0119] For convenience of description, circuit components arranged in the first group **1112a** of the second sub-array **1112** will be mainly described below. A multiplexer **MUX01** may output one of an output of the first sub-array **1111_7** or data loaded from the scratchpad memory **1200**, based on a second control signal. For example, in response to the second control signal having the first logical value (e.g., ‘0’), the multiplexer **MUX01** may output the data loaded from the scratchpad memory **1200**. For example, in response to the second control signal having the second logic value (e.g., ‘1’), the multiplexer **MUX01** may output an output of the first sub-array **1111_7**.

[0120] For example, when the second control signal has the first logic value (e.g., ‘0’), the non-accumulated operation may be performed in the direction of the input channel, and when the second control signal has the second logic value (e.g., ‘1’), the accumulated operation may be performed in the direction of the input channel.

[0121] The configuration, function, and operation of the multiplexer **MUX01** may correspond to the configuration, function, and operation of each of the other multiplexers (e.g., **MUX11**, **MUX81**, **MUX631**, etc.) connected to MAC operators **MAC1** to **MAC63**.

[0122] A MAC operator **MAC0** may be connected to the multiplexer **MUX01**. The MAC operator **MAC0** may perform a multiplication operation based on an output of the multiplexer **MUX01** and data of an input tensor. The data of the input tensor may be forwarded to other adjacent MAC operators (e.g., the MAC operator **MAC1**). The configuration, function, and operation of the MAC operator **MAC0** may correspond to the configuration, function, and operation of each of the other MAC operators (e.g., **MAC1**, **MAC8**, **MAC63**, etc.).

[0123] A multiplexer **MUX02** may be connected to an adder **ADD0**. The multiplexer **MUX02** may

transfer a result of the multiplication operation of the MAC operator **MAC0** to the adder **ADD0**, based on the second control signal. The configuration, function, and operation of the multiplexer **MUX02** may correspond to the configuration, function, and operation of each of similarly arranged multiplexers (e.g., **MUX12**, **MUX82**, **MUX632**, etc.).

[0124] The adder **ADD0** may be connected to a register **REG0**. The adder **ADD0** may sum a value stored in the register **REG0** and an output value of the multiplexer **MUX02**. The configuration, function, and operation of the adder **ADD0** may correspond to the configuration, function, and operation of each of similarly arranged adders (e.g., **ADD1**, **MUX8**, **MUX63**, etc.).

[0125] The register **REG0** may be connected to a multiplexer **MUX04**. The register **REG0** may store an output value of the adder **ADD0**. The register **REG0** may transfer the output value of the adder **ADD0** to the multiplexer **MUX04**. The configuration, function, and operation of the register **REG0** may correspond to the configuration, function, and operation of each of similarly arranged registers (e.g., **REG1**, **REG8**, **REG63**, etc.).

[0126] The multiplexer **MUX04** may or may not transfer the stored value of the register **REG0** to the scratchpad memory **1200**, based on a fourth control signal. The configuration, function, and operation of the multiplexer **MUX04** may correspond to the configuration, function, and operation of each of similarly arranged multiplexers (e.g., **MUX14**, **MUX84**, **MUX634**, etc.).

[0127] A multiplexer **MUX03** may be connected to a register **REG02**. The multiplexer **MUX03** may transfer the result of the multiplication operation of the MAC operator **MAC0** to the register **REG02**, based on the second control signal. The configuration, function, and operation of the multiplexer **MUX03** may correspond to the configuration, function, and operation of each of similarly arranged multiplexers (e.g., **MUX13**, **MUX83**, etc.).

[0128] The register **REG02** may store an output value of the multiplexer **MUX03**. The output value stored in the register **REG02** and an output value of a multiplexer **MUX13** of the adjacent MAC operator **MAC1** may be summed. A summation value may be summed again to an output value of a multiplexer of the next adjacent MAC operator **MAC2**.

[0129] When a non-accumulated operation is performed in the direction of the input channel, by the functions and operations of the above-described circuit elements, outputs of the nine MAC operators may be summed, and a summation value as a result of the summation may be transferred to the scratchpad memory **1200**. For example, in the case of the first group **1112a**, a value stored in a register **REG72** and an output of a multiplexer **MUX83** may be summed and transferred to the scratchpad memory **1200**.

[0130] The configuration, operation, and function of each of the second to seventh groups **1112b** to **1112g** correspond to the configuration, operation, and function of the above-described first group **1112a**, and thus descriptions thereof are omitted.

[0131] When the MAC operator **MAC63** that does not belong to the first to seventh groups **1112a** to **1112g** performs the non-accumulated operation in the direction of the input channel, data may not be mapped. However, when performing the accumulated operation in the direction of the input channel, the MAC operator **MAC63** may transfer a result value to the scratchpad memory **1200** by using a multiplexer **MUX632**, an adder **ADD63**, a register **REG631**, and a multiplexer **MUX634**.

[0132] FIG. **9** is a block diagram for explaining the configuration of a MAC operator **900** according to an embodiment of the disclosure. Matters of FIG. **9** that are the same as those described above with reference to FIGS. **1** to **8** will not be repeated herein. Referring to FIGS. **1** to **8** together, the configuration, operation, and function of each of the MAC operators included in the MAC array **1100**, the plurality of arrays **1110**, the first sub-arrays **1111**, and the second sub-array **1112** may correspond to the configuration, operation, and function of the MAC operator **900** of FIG. **9**.

[0133] The MAC operator **900** may include a multiplier **910**, an adder **ADD**, a precision converter **920**, an adder tree **930**, multiplexers **MUX1** to **MUX5**, and registers **REG_I**, **REG_W**, and **REG_O**.

[0134] The register **REG_I** may receive an input value. For example, the input value may include

bits with a predefined bit width. The register REG_I may store the input value. The register REG_I may be connected to the multiplier **910**. The register REG_I may transfer the stored input value to the multiplier **910**. The register REG_I may be connected to a multiplier of an adjacent MAC operator to forward the stored input value to the adjacent MAC operator.

[0135] The register REG_W may receive a weight value. For example, the weight value may include bits with a predefined bit width. The register REG_W may store the weight value. The register REG_W may be connected to the multiplier **910**. The register REG_W may transfer the stored weight value to the multiplier **910**.

[0136] The multiplier **910** may load the input value from the register REG_I and the weight value from the register REG_W. The multiplier **910** may perform a multiplication operation between the input value and the weight value. The multiplier **910** may be connected to the multiplexer MUX1 and the multiplexer MUX2. The multiplier **910** may support various precisions and datatypes. According to an embodiment of the disclosure, the multiplier **910** may operate flexibly according to precisions and datatypes of the input value and the weight value. The specific structure and operation of the multiplier **910** will be described in detail with reference to FIG. **10**.

[0137] The multiplexer MUX1 may or may not transfer an output of the multiplier **910** to the adder ADD, based on a third control signal. The third control signal may correspond to whether the datatypes of the input value and the weight value are integral types or floating-point types. For example, when the third control signal has a first logical value (e.g., '0'), the datatypes of the input value and the weight value may be floating-point types, and when the third control signal has a second logical value (e.g., '1'), the datatypes of the input value and the weight value may be integral types.

[0138] For example, when the third control signal has the first logic value (e.g., '0'), the multiplexer MUX1 may not transfer the output of the multiplier **910** to the adder ADD. For example, when the third control signal has the second logic value (e.g., '1'), the multiplexer MUX1 may transfer the output of the multiplier **910** to the adder ADD.

[0139] The multiplexer MUX2 may or may not transfer the output of the multiplier **910** to the precision converter **920**, based on the third control signal. For example, when the third control signal has the first logic value (e.g., '0'), the multiplexer MUX1 may transfer the output of the multiplier **910** to the precision converter **920**. For example, when the third control signal has the second logic value (e.g., '1'), the multiplexer MUX1 may not transfer the output of the multiplier **910** to the precision converter **920**.

[0140] The multiplexer MUX3 and the multiplexer MUX4 may receive a first partial sum, which is a partial sum output of the adjacent MAC operator. The multiplexer MUX3 and MUX4 may transfer the first partial sum to other structural elements, based on the third control signal. For example, when the third control signal has the first logical value (e.g., '0'), the multiplexer MUX4 may transfer the first partial sum to the adder tree **930**. For example, when the third control signal has the second logical value (e.g., '1'), the multiplexer MUX3 may transfer the first partial sum to the adder ADD.

[0141] The precision converter **920** may convert precision of the output of the multiplier **910**. For example, when the input value and the weight value are of floating-point types, a result of multiplication performed by the multiplier **910** may be converted to floating-point type 32-bit data through the precision converter **920**. According to an embodiment of the disclosure, accumulated errors of multiplication results may be prevented through the function of the precision converter **920**. The precision converter **920** may convert the output of the multiplier **910** to have a predefined bit width. The precision converter **920** may transfer a converted value to the adder tree **930**. The adder tree **930** may output a second partial sum by summing the first partial sum and an output of the precision converter **920**.

[0142] The adder ADD may sum the output of the multiplier **910** and the first partial sum. The multiplexer MUX5 may transfer the second partial sum to another adjacent MAC operator by

outputting an output of the adder ADD or an output of the adder tree **930**. For example, when the third control signal has the first logical value (e.g., '0'), the multiplexer MUX5 may output the output of the adder tree **930**. For example, when the third control signal has the second logical value (e.g., '1'), the multiplexer MUX5 may output the output of the adder **930**.

[0143] The register REG_O may store an output of the multiplexer MUX5. The register REG_O may transfer the stored output value, that is, the second partial sum, to another adjacent MAC operator. The other adjacent MAC operator may load the second partial sum from the register REG_O.

[0144] FIG. **10** is a block diagram for explaining the configuration of a multiplier **2000** according to an embodiment of the disclosure. Matters of FIG. **10** that are the same as those described above with reference to FIGS. **1** to **9** will not be repeated herein. A configuration, an operation, and a function of the multiplier **910** of FIG. **9** may correspond to a configuration, an operation, and a function of the multiplier **2000** of FIG. **10**. However, the disclosure is not limited thereto, and the multiplier **2000** may be disposed in a processor (e.g., a central processing unit (CPU), a graphics processing unit (GPU), or a neural processing unit (NPU)) by using various operators other than a MAC operator.

[0145] Referring to FIG. **10**, the multiplier **2000** may include multiplication logic **2100**, addition logic **2200**, XOR logic **2300**, a normalizer **2400**, a rounder **2500**, and a multiplexer MUX. However, not all of the illustrated components are essential. The multiplier **2000** may be implemented by more or less components than those illustrated in FIG. **10**.

[0146] The multiplication logic **2100** may include a plurality of sub-multiplication logics. Four sub-multiplication logics are shown in FIG. **10**, but the number of sub-multiplication logics is not limited thereto. The multiplication logic **2100** may output one result value or a plurality of result values based on a first input value and a second input value by using the sub-multiplication logics.

[0147] In an embodiment of the disclosure, the multiplication logic **2100** may be implemented as a carry-save multiplier (CSM), but the disclosure is not limited thereto.

[0148] In an embodiment of the disclosure, the multiplication logic **2100** may perform an $8b \times 8b$ multiplication operation, and each of the sub-multiplication logics may perform a $4b \times 4b$ multiplication operation. However, the disclosure is not limited thereto. In this case, the multiplication logic **2100** may perform one $8b \times 8b$ multiplication operation, two $4b \times 8b$ operations, two $8b \times 4b$ operations, or four $4b \times 4b$ operations.

[0149] Different data may be mapped to the multiplication logic **2100** according to precisions and datatypes of the first input value and the second input value. For example, the multiplication logic **2100** may perform multiplication between the first input value and the second input value, based on the datatypes of the first input value and the second input value being integral types. For example, the multiplication logic **2100** may perform multiplication between a mantissa of the first input value and a mantissa of the second input value, based on the datatypes of the first input value and the second input value being floating-point types.

[0150] The multiplication logic **2100** may be connected to the normalizer **2400** and the multiplexer MUX. The multiplication logic **2100** may transfer an operation result to the normalizer **2400** or to the multiplexer MUX.

[0151] The addition logic **2200** may include at least one sub-addition logic. Four sub-addition logics are shown in FIG. **10**, but the number of at least one sub-addition logic is not limited thereto. The at least one sub-addition logic may respectively correspond to at least one output value of the multiplication logic **2100**. In an embodiment of the disclosure, the number of at least one sub-addition logic may correspond to the maximum number of outputs of the multiplication logic **2100**. The at least one sub-addition logic may be arranged in the same number as the maximum number of at least one output of the multiplication logic **2100**. Each of the at least one sub-addition logic may be activated or deactivated according to the datatypes and precisions of the first input value and the second input value.

[0152] The addition logic **2200** may be deactivated based on the datatypes of the first input value and the second input value being integral types. The addition logic **2200** may sum an exponent of the first input value and an exponent of the second input value, based on the datatypes of the first input value and the second input value being floating-point types. Each of the first input value and the second input value may include a plurality of floating-point values. In this case, the addition logic **2200** may perform summation between an exponent of one of the plurality of floating-point values of the first input value and an exponent of one of the plurality of floating-point values of the second input value.

[0153] The addition logic **2200** may subtract a bias value mapped to a bit width of the exponent of the first input value and a bit width of the exponent of the second input value from a result of the summation. The bias value may correspond to bit widths of the precisions or exponents of the first and second input values. For example, the bias value may be $2^{\text{sup.}(\text{bit width of exponent})-1}$.

[0154] The addition logic **2200** may be connected to the normalizer **2400**. The addition logic **2200** may transfer an operation result to the normalizer **2400**.

[0155] The XOR logic **2300** may perform an exclusive OR operation. The XOR logic **2300** may be deactivated based on the datatypes of the first input value and the second input value being integral types. The XOR logic **2300** may perform an XOR operation between a sign of the first input value and a sign of the second input value, based on the datatypes of the first input value and the second input value being floating-point types.

[0156] The XOR logic **2300** may include at least one sub-XOR logic. Four sub-XOR logics are shown in FIG. **10**, but the number of at least one sub-XOR logic is not limited thereto. The at least one XOR addition logic may respectively correspond to at least one output value of the multiplication logic **2100**. In an embodiment of the disclosure, the number of at least one sub-XOR logic may correspond to the maximum number of outputs of the multiplication logic **2100**. The at least one sub-XOR logic may be arranged in the same number as the maximum number of at least one output of the multiplication logic **2100**. Each of the at least one sub-XOR logic may be activated or deactivated according to the datatypes and precisions of the first input value and the second input value.

[0157] The XOR logic **2300** may be connected to the normalizer **2400**. The XOR logic **2300** may transfer an operation result to the normalizer **2400**.

[0158] The normalizer **2400** may normalize and output input values. The normalizer **2400** may be deactivated based on the datatypes of the first input value and the second input value being integral types. The normalizer **2400** may normalize the output of the multiplication logic **2100** and the output of the addition logic **2200**, based on the datatypes of the first and second input values being floating-point types.

[0159] In an embodiment of the disclosure, the normalizer **2400** may include a plurality of sub-normalizers. FIG. **10** shows that the multiplication logic **2100** performs an $8b \times 8b$ operation and thus the normalizer **2400** includes five sub-normalizers, but the disclosure is not limited thereto. For example, one sub-normalizer may correspond to an $8b \times 8b$ multiplication operation result, and each of four sub-normalizers may correspond to a $4b \times 4b$ multiplication operation result. For example, when the multiplier **2000** performs one $8b \times 8b$ floating-point operation, one pre-specified sub-normalizer may be activated, and four unspecified sub-normalizers may be deactivated. For example, when the multiplier **2000** performs four $4b \times 4b$ floating-point operations, four pre-specified sub-normalizers may be activated, and one unspecified sub-normalizer may be deactivated.

[0160] The rounder **2500** may round off the input values with a predefined bit width and output a result of the rounding-off. The rounder **2500** may be deactivated based on the datatypes of the first input value and the second input value being integral types. The rounder **2500** may round off the input values with the predefined bit width by using the output of the normalizer **2400** and the output of the XOR logic **2300**, based on the datatypes of the first and second input values being

floating-point types. FIG. 10 shows that the multiplication logic **2100** performs an $8b \times 8b$ operation and thus the rounder **2500** includes five sub-rounders, but the disclosure is not limited thereto.

Whether the sub-rounders are activated corresponds to whether the sub-normalizers of the normalizer **2400** are activated, and thus, a description thereof is omitted.

[0161] The multiplexer MUX may output the output of the multiplication logic **2100**, based on the datatypes of the first input value and the second input value being integral types, and may output an output of the rounder **2500**, based on the datatypes of the first input value and the second input value being integral types.

[0162] In an embodiment of the disclosure, the multiplier **2000** may be controlled by a processor (not shown). The processor may obtain the first input value and the second input value. The processor may identify the datatypes and precisions of the first input value and the second input value. The processor may control each component of the multiplier **2000**, based on the identified datatypes and precisions.

[0163] In an embodiment of the disclosure, the multiplier **2000** may perform an $N\text{-bit} \times M\text{-bit}$ multiplication operation. In this case, each of the sub-multiplication logics of the multiplication logic **2100** may perform a $(N/p)\text{-bit} \times (M/q)\text{-bit}$ multiplication operation. In this case, N and M may be natural numbers divided by p and q , respectively. In this case, the multiplier **2000** may support multiplication operations such as $(N/p)\text{-bit} \times (M/q)\text{-bit}$, $(2N/p)\text{-bit} \times (M/q)\text{-bit}$, $\dots$, $N\text{-bit} \times (M/q)\text{-bit}$, $N\text{-bit} \times (2M/q)\text{-bit}$, $\dots$, and $N\text{-bit} \times M\text{-bit}$.

[0164] FIGS. 11A to 11C are conceptual diagrams for explaining operations of the multiplier **2000** according to an embodiment of the disclosure. Descriptions of FIGS. 11A to 11C that are the same as given above with reference to FIG. 10 will not be repeated herein. FIGS. 11A to 11C show that the multiplier **2000** supports an $8b \times 8b$ multiplication operation, but the disclosure is not limited thereto. The multiplier **2000** may support multiplication operations with respect to various bit widths.

[0165] Referring to FIG. 11A, it is assumed that a first input value and a second input value have a datatype and precision of bfloat16. bfloat16 is a floating-point type, with 1 bit corresponding to a sign, 8 bits corresponding to an exponent, and 7 bits corresponding to a mantissa. Because bfloat16 is the floating-point type, the addition logic **2200**, the XOR logic **2300**, the normalizer **2400**, and the rounder **2500**, including the multiplication logic **2100**, may be activated.

[0166] All sub-multiplication logics of the multiplication logic **2100** may be activated. Because the mantissa additionally includes 1 bit as an implicit bit, an operation may be performed on the mantissa with 8 bits rather than 7 bits. The multiplication logic **2100** may output a multiplication operation between the 8-bit mantissa of the first input value and the 8-bit mantissa of the second input value.

[0167] Because the multiplication logic **2100** outputs one $8b \times 8b$ operation result, one of the sub-addition logics of the addition logic **2200** may be activated. The addition logic **2200** may perform exponential addition between the 8-bit exponent of the first input value and the 8-bit exponent of the second input value, receive a bias value of $2^{\text{sup.}(8)} - 1$, and output a value obtained by subtracting the bias value from a result of the exponential addition.

[0168] Because the multiplication logic **2100** outputs one $8b \times 8b$ operation result, one of the sub-addition logics of the addition logic **2300** may be activated. The XOR logic **2300** may perform an XOR operation between the 1-bit sign of the first input value and the 1-bit sign of the second input value.

[0169] Because the multiplication logic **2100** outputs one $8b \times 8b$ operation result, one of the sub-normalizers of the normalizer **2400** may be activated. The normalizer **2400** may perform normalization on an output of the multiplication logic **2100** and an output of the addition logic **2200**.

[0170] Because the multiplication logic **2100** outputs one $8b \times 8b$ operation result, one of the sub-rounders of the rounder **2500** may be activated. The rounder **2500** may receive an output of the

normalizer **2400** and an output of the XOR logic **2300**. The rounder **2500** may round off the output of the normalizer **2400** with a predefined bit width. The rounder **2500** may connect the output of the XOR logic **2300** to a rounded value and transfer the output to the multiplexer MUX.

[0171] The multiplexer MUX may output an output of the rounder **2500**, based on the datatypes of the first input value and the second input value being floating-point types.

[0172] Referring to FIG. 11B, it is assumed that a first input value and a second input value have a datatype and precision of FP8. FP8 is a floating-point type, with 1 bit corresponding to a sign, 4 bits corresponding to an exponent, and 3 bits corresponding to a mantissa. Accordingly, the first input value may include four FP8 floating-point values, and the second input value may include four FP8 floating-point values. Because FP8 is the floating-point type, the addition logic **2200**, the XOR logic **2300**, the normalizer **2400**, and the rounder **2500**, including the multiplication logic **2100**, may be activated.

[0173] All sub-multiplication logics of the multiplication logic **2100** may be activated. Because the mantissa additionally includes 1 bit as an implicit bit, an operation may be performed on the mantissa with 4 bits rather than 3 bits. Each of the activated sub-multiplication logics may output a multiplication operation between the 4-bit mantissa of each of the four FP8 values included in the first input value and the 4-bit mantissa of each of the four FP8 values included in the second input value.

[0174] Because the multiplication logic **2100** outputs four 4b×4b operation results, four of the sub-addition logics of the addition logic **2200** may be activated. Each of the activated sub-addition logics may perform exponential addition between the 4-bit exponent of each of the four FP8 values included in the first input value and the 4-bit exponent of each of the four FP8 values included in the second input value, receive a bias value of $22.\text{sup.}(4)-1$, and output a value obtained by subtracting the bias value from a result of the exponential addition.

[0175] Because the multiplication logic **2100** outputs four 4b×4b operation results, four of the sub-XOR logics of the XOR logic **2300** may be activated. Each of the activated sub-XOR logics may perform an XOR operation between the 1-bit sign of each of the four FP8 values included in the first input value and the 1-bit sign of each of the four FP8 values included in the second input value.

[0176] Because the multiplication logic **2100** outputs four 4b×4b operation results, four of the sub-normalizers of the normalizer **2400** may be activated. The normalizer **2400** may perform normalization on an output of the multiplication logic **2100** and an output of the addition logic **2200**.

[0177] Because the multiplication logic **2100** outputs four 4b×4b operation results, four of the sub-rounders of the rounder **2500** may be activated. The rounder **2500** may receive an output of the normalizer **2400** and an output of the XOR logic **2300**. The rounder **2500** may round off the output of the normalizer **2400** with a predefined bit width. The rounder **2500** may connect the output of the XOR logic **2300** to a rounded value and transfer the output to the multiplexer MUX.

[0178] The multiplexer MUX may output an output of the rounder **2500**, based on the datatypes of the first input value and the second input value being floating-point types.

[0179] Referring to FIG. 11C, it is assumed that a first input value and a second input value have a datatype and precision of INT8. INT8 is an integral datatype with a bit width of 8 bits. Because INT8 is an integer type, only the multiplication logic **2100** may be activated, and the addition logic **2200**, the XOR logic **2300**, the normalizer **2400**, and the rounder **2500** may be deactivated.

[0180] All sub-multiplication logics of the multiplication logic **2100** may be activated. The multiplication logic **2100** may output a multiplication operation between the 8-bit integer corresponding to the first input value and the 8-bit integer corresponding to the second input value by using the sub-multiplication logics.

[0181] The multiplexer MUX may output an output of the multiplication logic **2100**, based on the datatypes of the first input value and the second input value being integral types.

[0182] FIG. 12 is a block diagram for explaining the configuration of the multiplication logic 2100 according to an embodiment of the disclosure. Matters of FIG. 12 that are the same as those described above with reference to FIGS. 10 to 11C will not be repeated herein.

[0183] Referring to FIG. 10 along with FIG. 12, the multiplication logic 2100 may include four sub-multiplication logics, namely, first, second, third, and fourth sub-multiplication logics 2110_1, 2110_2, 2110_3, and 2110_4. However, the disclosure is not limited thereto, and the multiplication logic 2100 may include fewer or more sub-multiplication logics than the four sub-multiplication logics.

[0184] Each of the first, second, third, and fourth sub-multiplication logics 2110_1, 2110_2, 2110_3, and 2110_4 may be implemented as a 4b×4b multiplier, or may be implemented as a 5b×5b multiplier including a sign bit. FIG. 12 shows that each of the first, second, third, and fourth sub-multiplication logics 2110_1, 2110_2, 2110_3, and 2110_4 performs the 5b×5b multiplication operation, but the disclosure is not limited thereto.

[0185] Each of the first, second, third, and fourth sub-multiplication logics 2110_1, 2110_2, 2110_3, and 2110_4 may include at least one shifter logic SHIFT. The shifter logic may shift an output of each of the first, second, third, and fourth sub-multiplication logics 2110_1, 2110_2, 2110_3, and 2110_4 by a predefined number of bits. For example, one shifter logic SHIFT may perform 4-bit left shifting. However, the disclosure is not limited thereto, and shifting bits and/or a shifting direction of the shifter logic SHIFT may vary.

[0186] In an embodiment of the disclosure, two shifter logics may be connected to the first sub-multiplication logic 2110_1. The two shifter logics may perform left shifting on an output of the first sub-multiplication logic 2110_1 by 8 bits. One shifter logic may be connected to the second sub-multiplication logic 2110_2. The one shifter logic may perform left shifting on an output of the second sub-multiplication logic 2110_2 by 4 bits. One shifter logic may be connected to the third sub-multiplication logic 2110_3. The one shifter logic may perform left shifting on an output of the third sub-multiplication logic 2110_3 by 4 bits.

[0187] In an embodiment of the disclosure, the multiplication logic 2100 may include a first multiplexer logic 2121 and a second multiplexer logic 2122. Each of the first, second, third, and fourth sub-multiplication logics 2110_1, 2110_2, 2110_3, and 2110_4 may be connected to the first multiplexer logic 2121 and the second multiplexer logic 2122.

[0188] The first multiplexer logic 2121 may receive a control signal indicating whether summation of the outputs of the first, second, third, and fourth sub-multiplication logics 2110_1, 2110_2, 2110_3, and 2110_4 is necessary based on precisions and datatypes of the values input to the multiplication logic 2100. The first multiplexer logic 2121 may transfer an output of the multiplication logic 2100 to the selective adder ADD, based on the control signal indicating that summation of the outputs of the first, second, third, and fourth sub-multiplication logics 2110_1, 2110_2, 2110_3, and 2110_4 is necessary. The selective adder ADD may output a first output of 18 bits by summing the outputs of the selective adder ADD may output a first output of 18 bits by summing the outputs of the first, second, third, and fourth sub-multiplication logics 2110_1, 2110_2, 2110_3, and 2110_4.

[0189] The multiplexer logic 2121 may output a second output of 40 bits that is a set of the outputs of the first, second, third, and fourth sub-multiplication logics 2110_1, 2110_2, 2110_3, and 2110_4, based on the control signal indicating that summation of the outputs of the first, second, third, and fourth sub-multiplication logics 2110_1, 2110_2, 2110_3, and 2110_4 is unnecessary.

[0190] FIG. 13 is a conceptual diagram illustrating a multiplication operation according to an embodiment of the disclosure.

[0191] Referring to FIG. 12 along with FIG. 13, it is assumed that a weight value W is 8 bits and an input value X is 8 bits. The weight value W may be divided into a first weight sub-word w0 of four bits and a second weight sub-word w1 of four bits. The input value X may be divided into a first input sub-word x0 of four bits and a second input sub-word x1 of four bits. To perform a

multiplication operation between 8 bits and 8 bits, a 16-bit result value may be obtained by performing a multiplication operation between 4-bit sub-words and then performing shifting and summation.

[0192] For example, the fourth sub-multiplication logic **2110_4** may obtain a first result value by performing multiplication between the first weight sub-word **w0** and the first input sub-word **x0**. The third sub-multiplication logic **2110_3** may obtain a second result value by performing multiplication between the second weight sub-word **w1** and the first input sub-word **x0**. The second sub-multiplication logic **2110_2** may obtain a third result value by performing multiplication between the first weight sub-word **w0** and the second input sub-word **x1**. The first sub-multiplication logic **2110_1** may obtain a fourth result value by performing multiplication between the second weight sub-word **w1** and the second input sub-word **x1**. A shifter logic may perform left shifting on the second result value and the third result value by 4 bits. The shifter logic may perform left shifting on the fourth result value by 8 bits.

[0193] The multiplication logic **2100** may output a 16-bit result value, based on the first result value and shifted second to fourth result values.

[0194] FIGS. **14A** and **14B** are block diagrams for explaining operations of a multiplication operation according to an embodiment of the disclosure. Matters of FIGS. **14A** and **14B** that are the same as those described above with reference to FIGS. **10** through **13** will not be repeated herein.

[0195] Referring to FIG. **14A**, it is assumed that the multiplication logic **2100** performs a multiplication operation between a first input value of INT8 and a second input value of INT8. Each of the sub-multiplication logic may perform a multiplication operation between 4-bit sub-words. At least one shifting logic may shift at least a portion of the multiplication operation by 4 bits or 8 bits. When a datatype of an input value is INT8, a selective adder may be activated, and result values of the sub-multiplication logics may be summed and output.

[0196] In an embodiment of the disclosure, it is assumed that the multiplication logic **2100** performs the multiplication operation between the first input value of bfloat16 and the second input value of bfloat16. Each of the sub-multiplication logics may perform the multiplication operation between 4-bit sub-words among mantissa values of 8-bit bfloat16. The at least one shifting logic may shift at least a portion of the multiplication operation by 4 bits or 8 bits. When the datatype of the input value is bfloat16, the selective adder may be activated, and result values of the sub-multiplication logics may be summed and output. However, the disclosure is not limited thereto, and the selective adder may be deactivated by predefined settings, regardless of the datatype of the input value.

[0197] Referring to FIG. **14B**, it is assumed that the multiplication logic **2100** performs a multiplication operation between a first input value of INT4 and a second input value of INT4. Each of the sub-multiplication logics may perform a multiplication operation between one of a plurality of 4-bit INT4 integer values included in the first input value and one of a plurality of 4-bit INT4 integer values included in the second input value. The at least one shifting logic may be deactivated. When the datatype of the input value is IN4, the selective adder may be deactivated, and result values of the sub-multiplication logics may be output without being summed.

[0198] In an embodiment of the disclosure, it is assumed that the multiplication logic **2100** performs the multiplication operation between the first input value of FP8 and the second input value of FP8. Each of the sub-multiplication logics may perform a multiplication operation between one of a plurality of 4-bit FP8 mantissa values included in the first input value and one of a plurality of 4-bit FP8 mantissa values included in the second input value. The at least one shifting logic may be deactivated. When the datatype of the input value is FP8, the selective adder may be deactivated, and result values of the sub-multiplication logics may be output without being summed. However, the disclosure is not limited thereto, and the selective adder may be activated by predefined settings, regardless of the datatype of the input value.

[0199] FIG. **15** is a block diagram for explaining the configuration of the addition logic **2200**

according to an embodiment of the disclosure. Matters of FIG. 15 that are the same as those described above with reference to FIGS. 10 to 14B will not be repeated herein.

[0200] Referring to FIG. 10 along with FIG. 15, the addition logic 2200 may include four sub-addition logics, namely, first, second, third, and fourth sub-multiplication logics 2210_1, 2210_2, 2210_3, and 2210_4. However, the disclosure is not limited thereto, and the addition logic 2200 may include fewer or more sub-addition logics than the four sub-addition logics.

[0201] Each of a first input value and a second input value may include at least one exponent according to a datatype and precision thereof. For example, when datatypes and precisions of the first input value and the second input value are bfloat16, each of the first input value and the second input value may include one exponent. In this case, only the first sub-addition logic 2210_1 may be activated.

[0202] For example, when the datatypes and precisions of the first input value and the second input value are FP8, each of the first input value and the second input value may include four exponents (e.g., first to fourth exponents). In this case, the first to fourth sub-addition logics 2210_1 to 2210_4 may be activated.

[0203] For convenience of description, the configuration, operation, and function of the first sub-addition logic 2210_1 are mainly described below.

[0204] The first sub-addition logic 2210_1 may include a first unsigned adder ADD11 and a second unsigned adder ADD12. In an embodiment of the disclosure, the first unsigned adder ADD11 and the second unsigned adder ADD12 may each be implemented as a CSA, but the disclosure is not limited thereto.

[0205] The first unsigned adder ADD11 may receive a first exponent of the first input value and a first exponent of the second input value. The first unsigned adder ADD11 may sum the first exponent of the first input value and the first exponent of the second input value. The first unsigned adder ADD11 may transfer a result of the summation to the second unsigned adder ADD12.

[0206] The second unsigned adder ADD12 may subtract a predefined bias value from the result of the summation. The bias value may vary depending on the precisions and datatypes of the first input value and the second input value. The second unsigned adder ADD12 may output a subtraction result (e.g., first exponent output).

[0207] The configuration, operation, and function of the first sub-addition logic 2210_1 may respectively correspond to the configuration, operation, and function of each of the second to fourth sub-addition logics 2210_2 to 2210_4. The configuration, operation, and function of each of the first unsigned adder ADD11 and the second unsigned adder ADD12 may respectively correspond to the configuration, operation, and function of each of first unsigned adders ADD21, ADD31, and ADD41 and second unsigned adders ADD22, ADD32, and ADD42 of the other second to fourth sub-addition logics 2210_2 to 2210_4.

[0208] FIG. 16 is a block diagram for explaining the configuration of the first sub-addition logic 2210_1 according to an embodiment of the disclosure. Matters of FIG. 16 that are the same as those described above with reference to FIGS. 10 to 15 will not be repeated herein. For convenience of description, the configuration of the first sub-addition logic 2210_1 in FIG. 15 among the sub-addition logics is described as an example.

[0209] Referring to FIG. 15 together with FIG. 16, the first unsigned adder ADD11 may include a plurality of full adders FA and a half adder HA. FIG. 16 shows that the first unsigned adder ADD11 includes 7 full adders FA, but the disclosure is not limited thereto. The first unsigned adder ADD11 may include fewer or more full adders FA than the 7 full adders FA.

[0210] According to an embodiment of the disclosure, as the number of full adders FA increases, the precision of an exponent that may be processed by the first sub-addition logic 2210_1 may increase. When a sum of the number of full adders FA included in the first unsigned adder ADD11 and the number of half adders HA included in the first unsigned adder ADD11 is N, the first sub-multiplication logic 2210_1 in FIG. 15 may support an addition operation on exponents of N bits or

less. For example, a combination of the plurality of full adders FA and half adders HA shown in FIG. 16 may support addition operations on exponents of 8 bits or less.

[0211] The half adder HA may output a first carry bit C0 and a first summation bit S0 by using a first bit E.sub.A0 of a first exponent of a first input value and a first bit E.sub.B0 of a first exponent of a second input value as inputs.

[0212] Each of the plurality of full adders FA may output a next carry bit and a next summation bit by using a previous carry bit, a predefined bit of the first exponent of the first input value, and a predefined bit of the first exponent of the second input value as inputs.

[0213] The plurality of full adders FA may include first to seventh full adders. For example, the first full adder may output a second carry bit C1 and a second summation bit S1 by using the first carry bit C0, a second bit E.sub.A1 of the first exponent of the first input value, and a second bit E.sub.B1 of the first exponent of the second input value as inputs. For example, the second full adder may output a third carry bit C2 and a third summation bit S2 by using the second carry bit C1, a third bit E.sub.A2 of the first exponent of the first input value, and a third bit E.sub.B2 of the first exponent of the second input value as inputs. For example, the third full adder may output a fourth carry bit C3 and a fourth summation bit S3 by using the third carry bit C2, a fourth bit E.sub.A3 of the first exponent of the first input value, and a fourth bit E.sub.B3 of the first exponent of the second input value as inputs. For example, the fourth full adder may output a fifth carry bit C4 and a fifth summation bit S4 by using the fourth carry bit C3, a fifth bit E.sub.A4 of the first exponent of the first input value, and a fifth bit E.sub.B4 of the first exponent of the second input value as inputs. For example, the fifth full adder may output a sixth carry bit C5 and a sixth summation bit S5 by using the fifth carry bit C4, a sixth bit E.sub.A5 of the first exponent of the first input value, and a sixth bit E.sub.B5 of the first exponent of the second input value as inputs. For example, the sixth full adder may output a seventh carry bit C6 and a seventh summation bit S6 by using the sixth carry bit C5, a seventh bit E.sub.A6 of the first exponent of the first input value, and a seventh bit E.sub.B6 of the first exponent of the second input value as inputs. For example, the seventh full adder may output an eighth carry bit C7 and an eighth summation bit S7 by using the seventh carry bit C6, an eighth bit E.sub.A7 of the first exponent of the first input value, and an eighth bit E.sub.B7 of the first exponent of the second input value as inputs.

[0214] The second unsigned adder ADD12 may include a plurality of full adders FA and a half adder HA. FIG. 16 shows that the second unsigned adder ADD12 includes 8 full adders FA, but the disclosure is not limited thereto. The first unsigned adder ADD11 may include fewer or more full adders FA than the 8 full adders FA. The second unsigned adder ADD12 may include one more full adder FA than the first unsigned adder ADD11.

[0215] The half adder HA of the second unsigned adder ADD12 may output the first carry bit C0 and the first summation bit S0 by using the first summation bit S0 output by the half adder HA of the first unsigned adder ADD11 and a first bit B0 of a bias value as inputs. The first summation bit S0 output by the half adder HA of the first unsigned adder ADD11 and the first summation bit S0 output by the half adder HA of the second unsigned adder ADD12 may refer to different bits.

[0216] Each of the plurality of full adders FA may output a next carry bit and a next summation bit by using a previous carry bit, a predefined bit of the bias value, and a corresponding summation bit among summation bits output by the full adders FA of the first unsigned adder ADD11 as inputs.

[0217] The plurality of full adders FA may include eighth to fifteenth full adders. For example, the eighth full adder may output the second carry bit C1 and the second summation bit S1 by using the first carry bit C0, the second summation bit S1 output by the first full adder, and a second bit B1 of the bias value as inputs. For example, the ninth full adder may output the third carry bit C2 and the third summation bit S2 by using the second carry bit C1, the third summation bit S2 output by the second full adder, and a third bit B2 of the bias value as inputs. For example, the tenth full adder may output the fourth carry bit C3 and the fourth summation bit S3 by using the third carry bit C2, the fourth summation bit S3 output by the third full adder, and a fourth bit B3 of the bias value as

inputs. For example, the eleventh full adder may output the fifth carry bit C4 and the fifth summation bit S4 by using the fourth carry bit C3, the fifth summation bit S4 output by the fourth full adder, and a fifth bit B4 of the bias value as inputs. For example, the twelfth full adder may output the sixth carry bit C5 and the sixth summation bit S5 by using the fifth carry bit C4, the sixth summation bit S5 output by the fifth full adder, and a sixth bit B5 of the bias value as inputs. For example, the thirteenth full adder may output the seventh carry bit C6 and the seventh summation bit S6 by using the sixth carry bit C5, the seventh summation bit S6 output by the sixth full adder, and a seventh bit B6 of the bias value as inputs. For example, the fourteenth full adder may output the eighth carry bit C7 and the eighth summation bit S7 by using the seventh carry bit C6, the eighth summation bit S7 output by the seventh full adder, and an eighth bit B7 of the bias value as inputs. For example, the fifteenth full adder may output a ninth summation bit S8 and a tenth summation bit S9 by using the eighth carry bit C7, the eighth carry bit C7 output by the seventh full adder, and a ninth bit B8 of the bias value as inputs.

[0218] The summation bits S0 to S9, which are outputs of the second unsigned adder ADD12, may be used as output values of the addition logic 2200 with the first exponent of the first input value and the second exponent of the second input value as inputs.

[0219] FIG. 17 is a block diagram for explaining the configuration of the XOR logic 2300 according to an embodiment of the disclosure. Matters of FIG. 17 that are the same as those described above with reference to FIGS. 10 to 16 will not be repeated herein.

[0220] Referring to FIG. 10 along with FIG. 17, the addition logic 2300 may include four sub-XOR logics, namely, first, second, third, and fourth sub-XOR logics 2310_1, 2310_2, 2310_3, and 2310_4. However, the disclosure is not limited thereto, and the XOR logic 2300 may include fewer or more sub-XOR logics than the four sub-XOR logics.

[0221] Each of a first input value and a second input value may include at least one sign according to a datatype and precision thereof. For example, when datatypes and precisions of the first input value and the second input value are bfloat16, each of the first input value and the second input value may include one sign. In this case, only the first sub-XOR logic 2310_1 may be activated.

[0222] For example, when the datatypes and precisions of the first input value and the second input value are FP8, each of the first input value and the second input value may include four signs (e.g., first to fourth signs). In this case, the first to fourth sub-XOR logics 2310_1 to 2310_4 may be activated.

[0223] For convenience of description, the configuration, operation, and function of the first sub-XOR logic 2310_1 are mainly described below.

[0224] The first sub XOR logic 2310_1 may receive a first sign of the first input value and a first sign of the second input value. The first sub XOR logic 2310_1 may perform an XOR operation by using the first sign of the first input value and the first sign of the second input value. For example, when a bit corresponding to the first sign of the first input value and a bit corresponding to the first sign of the second input value are the same, the first sub XOR logic 2310_1 may output a first value, and when the bit corresponding to the first sign of the input value and the bit corresponding to the first sign of the second input value are different, the first sub XOR logic 2310_1 may output a second value. For example, when the first sign of the first input value represents a positive number and the first sign of the second input value represents a positive number, the first sub-XOR logic 2310_1 may output a sign value representing a positive number. For example, when the first sign of the first input value represents a positive number and the first sign of the second input value represents a negative number, the first sub-XOR logic 2310_1 may output a sign value representing a negative number.

[0225] FIG. 18 is a block diagram for explaining the configuration of an electronic device 3000 according to an embodiment of the disclosure. Matters of FIG. 18 that are the same as those described above with reference to FIGS. 1 to 17 will not be repeated herein.

[0226] Referring to FIG. 18, the electronic device 3000 may include a hardware accelerator 3100, a

processor **3200**, a memory **3300**, and an input/output interface **3400**. However, the components of the electronic device **3000** are not limited to the above-described examples, and the electronic device **3000** may include more or fewer components than the aforementioned components. [0227] In an embodiment of the disclosure, at least some of the hardware accelerator **3100**, the processor **3200**, the memory **3300**, and the input/output interface **3400** may be implemented in the form of a single chip, and the processor **3200** may include one or more processors. The electronic device **3000** may be a device such as a PC, laptop, or server, as well as a mobile device such as a smartphone.

[0228] The configuration, function, and operation of the hardware accelerator **3100** may respectively correspond to the configuration, function, and operation of the hardware accelerator **1000** described above with reference to FIG. **1**. Descriptions of FIG. **18** that is the same as given above with reference to FIG. **1** will not be repeated herein.

[0229] The hardware accelerator **3100** may perform a matrix operation between a first tensor (e.g., an input tensor) and a second tensor (e.g., a weight tensor) by using a plurality of MAC operators.

[0230] The hardware accelerator **3100** may perform learning or inference of a DNN model under the control by the processor **3200**. The hardware accelerator **3100** may read data (e.g., the first tensor and the second tensor) stored in the memory **3300** to perform an operation. An operation result of the hardware accelerator **3100** may be stored in the memory **3300**.

[0231] In an embodiment of the disclosure, the hardware accelerator **3100** may operate in a first operation mode in which operation results of the plurality of MAC operators are output without being accumulated in a direction of an input channel, or in a second operation mode in which the operation results of the plurality of MAC operators are accumulated in the direction of the input channel and the accumulated operation results are output.

[0232] The processor **3200** is configured to control a series of processes to operate the hardware accelerator **3100** according to the embodiments described above with reference to FIGS. **1** to **17**, and may include one or more processors. In this case, the one or more processors may be general-purpose processors such as a CPU, an AP, or a digital signal processor (DSP).

[0233] The processor **3200** may write data to the memory **3300** or read data stored in the memory **3300**, and, in particular, may execute a program stored in the memory **3300** to process data according to a predefined operation rule or a DNN model. In an embodiment, the processor **3200** may control the hardware accelerator **3100**, based on DNN information including at least one of the number of layers of the DNN, types of layers, shapes of tensors, dimensions of tensors, operation modes, bit precisions, types of batch normalizations, types of pooling layers, or types of ReLU functions.

[0234] In an embodiment of the disclosure, the processor **3200** may control the hardware accelerator **3100** to perform an operation of at least one DNN model, based on a user input. The processor **3200** may control the hardware accelerator **3100** to perform an operation of at least one DNN model, based on a plurality of user inputs. When an operation on the same DNN model is requested from another user, the processor **3200** may allow the hardware accelerator **3100** to simultaneously perform operations on the same DNN models based on user inputs corresponding to the user request. According to an embodiment of the disclosure, in a multitenant environment, the hardware accelerator **3100** may simultaneously process requests from other users with respect to operations of the same DNN model, by grouping or not grouping MAC operators of the MAC array.

[0235] In an embodiment of the disclosure, the hardware accelerator **3100** may include a multiplier **3110**. The configuration, function, and operation of the multiplier **3110** may respectively correspond to the configuration, function, and operation of the multiplier **2000** described above with reference to FIG. **10**. Descriptions of FIG. **18** that is the same as given above with reference to FIG. **10** will not be repeated herein.

[0236] Operations of the multiplier **3110** may be implemented by the processor **3200**. The

processor **3200** may obtain a first input value and a second input value from the input/output interface **3400** or the memory **3300**. The processor **3200** may identify datatypes and precisions of the first input value and the second input value. For example, the datatype may be an integral type or a floating-point type, but the disclosure is not limited thereto. For example, the precision may indicate a bit width of the first input value or the second input value. For example, the first input value and/or the second input value may be INT4, INT8, bfloat16, FP8, FP32, etc. However, the disclosure is not limited thereto, and the first input value and/or the second input value may have various datatypes and/or precisions.

[0237] In an embodiment of the disclosure, the processor **3200** may distribute bits of the first input value and bits of the second input value to sub-multiplication logics of multiplication logic of the multiplier **3110**, based on the identified datatypes and precisions. The processor **3200** may obtain at least one output of the multiplication logic, based on outputs of the sub-multiplication logics.

[0238] In an embodiment of the disclosure, the processor **3200** may determine whether to activate at least one shifting logic of the multiplication logic, based on the identified datatypes and precisions. The at least one activated shifting logic may shift at least one of the outputs of the sub-multiplication logics.

[0239] In an embodiment of the disclosure, the processor **3200** may determine whether to activate a selective adder of the multiplication logic, based on the identified datatypes and precisions. The activated selective adder may sum the outputs of the sub-multiplication logics.

[0240] In an embodiment of the disclosure, the processor **3200** may distribute bits corresponding to a mantissa of the first input value to the sub-multiplication logics, based on the identified datatypes being floating-point types. The processor **3200** may distribute bits corresponding to a mantissa of the second input value to the sub-multiplication logics, based on the identified datatypes being integral types.

[0241] In an embodiment of the disclosure, the processor **3200** may determine whether to activate an addition logic of the multiplier **3110**, based on the identified datatypes and precisions. The activated addition logic may sum bits corresponding to an exponent of the second input value and bits corresponding to an exponent of the first input value. The activated addition logic may subtract bias values mapped to the identified datatypes and precisions from a result of the summation.

[0242] In an embodiment of the disclosure, the processor **3200** may determine whether to activate an XOR logic of the multiplier **3110**, based on the identified datatypes and precisions. The activated XOR logic may perform an XOR operation between a bit corresponding to a sign of the first input value and a bit corresponding to a sign of the second input value.

[0243] In an embodiment of the disclosure, the processor **3200** may determine whether to activate a normalizer, based on the identified datatypes and precisions. The activated normalizer may normalize at least one output of the multiplication logic and at least one output of the addition logic.

[0244] In an embodiment of the disclosure, the processor **3200** may determine whether to activate a rounder, based on the identified datatypes and precisions. The activated rounder may perform rounding with a predefined bit width by using at least one output of the XOR logic and at least one output of the normalizer.

[0245] In an embodiment of the disclosure, the processor **3200** may determine at least one output of the multiplication logic to be the output of the multiplier **3110**, based on the identified datatypes being integral types. The processor **3200** may determine the output of the normalizer to be the output of the multiplier **3110**, based on the identified datatypes being floating-point types.

[0246] The hardware accelerator **3100**, the multiplier **3110**, and the processor **3200** may perform operations described in the above-described embodiments, and the operations that are described as being performed by the electronic device **3000** in the above-described embodiments may be regarded as being performed by the hardware accelerator **3100**, the multiplier **3110**, and the processor **3200** unless otherwise stated.

[0247] The memory **3300**, which is a component for storing various programs or data, may be composed of storage media, such as read-only memory (ROM), random access memory (RAM), hard disks, compact disc (CD)-ROM, and digital versatile discs (DVDs), or a combination thereof. The memory **3300** may not exist separately but may be included in the processor **3200** or the hardware accelerator **3100**. The memory **3300** may be implemented as a volatile memory, a non-volatile memory, or a combination of a volatile memory and a non-volatile memory. The memory **3300** may store programs for performing operations performed by the hardware accelerator **3100** or the processor **3200**. The memory **3300** may provide stored data to the hardware accelerator **3100** or the processor **3200** according to a request from the hardware accelerator **3100** or the processor **3200**. In an embodiment, the memory **3300** may store at least one instruction executed by the processor **3200**. The memory **3300** may store parameters or hyperparameters used to train or infer a DNN.

[0248] The input/output interface **3400** may include an input interface (e.g., a touch screen, a hard button, or a microphone) for receiving control commands or information from a user, and an output interface (e.g., a display panel or a speaker) for displaying an execution result of an operation under the user's control or a state of the electronic device **3000**. In an embodiment of the disclosure, the electronic device **3000** may receive hyperparameters of the DNN model and/or a user input requesting the hardware accelerator **3100** for an operation of the DNN model, through the input/output interface **3400**.

[0249] FIG. **19** is a flowchart for explaining the operation of a multiplier according to an embodiment of the disclosure. Matters of FIG. **19** that are the same as those described above with reference to FIGS. **1** to **18** will not be repeated herein.

[0250] In an embodiment of the disclosure, operations **S1910** to **S1940** may be performed by at least one of the electronic device **3000**, the multiplier **3110**, the hardware accelerator **3100**, and the processor **3200** of FIG. **18**. However, the disclosure is not limited thereto, and operations **S1910** to **S1940** may be performed by any electronic device. An operating method of the multiplier or a controlling method of the multiplier according to an embodiment of the disclosure is not limited to that shown in FIG. **19**, and may omit at least one of the operations shown in FIG. **19** or may further include an operation not shown in FIG. **19**.

[0251] In operation **S1910**, the electronic device **3000** may obtain a first input value and a second input value. In an embodiment of the disclosure, each of the first input value and the second input value may include a plurality of bits having the same datatype and precision. For example, the first input value may correspond to training data or inference target data. The first input value may represent a specific value of an input tensor. For example, the second input value may correspond to weight data. The second input value may represent a specific value of a weight tensor.

[0252] In operation **S1920**, the electronic device **3000** may identify datatypes and precisions of the first input value and the second input value. The electronic device **3000** may transfer a control signal mapped according to the datatypes and precisions of the first input value and the second input value to the multiplier. The multiplier may distribute data to components of the multiplier and/or activate/deactivate the components of the multiplier, based on the control signal.

[0253] In operation **S1930**, the electronic device **3000** may distribute bits of the first input value and bits of the second input value to sub-multiplication logics of multiplication logic included in the multiplier, based on the identified datatypes and precisions. In an embodiment of the disclosure, the sub-multiplication logics may perform a $4b \times 4b$ multiplication operation that does not process signs, or a $5b \times 5b$ multiplication operation that processes signs.

[0254] In operation **S1940**, the electronic device **3000** may obtain at least one output of the multiplication logic, based on outputs of the sub-multiplication logics. In an embodiment of the disclosure, the electronic device **3000** may perform shifting on at least one of the outputs of the sub-multiplication logics. In an embodiment of the disclosure, the electronic device **3000** may sum the outputs of the sub-multiplication logics.

[0255] FIG. 20 is a flowchart for explaining the operation of shifting logic according to an embodiment of the disclosure. Matters of FIG. 20 that are the same as those described above with reference to FIGS. 1 to 19 will not be repeated herein.

[0256] In an embodiment of the disclosure, operations S2010 and S2020 may be performed by at least one of the electronic device 3000, the multiplier 3110, the hardware accelerator 3100, and the processor 3200 of FIG. 18. However, the disclosure is not limited thereto, and operations S2010 and S2020 may be performed by any electronic device. An operating method of the shifting logic or a controlling method of the shifting logic according to an embodiment of the disclosure is not limited to that shown in FIG. 20, and may omit at least one of the operations shown in FIG. 20 or may further include an operation not shown in FIG. 20.

[0257] In an embodiment of the disclosure, operations S2010 and S2020 may be performed by the electronic device 3000 between operations S1930 and S1940 of FIG. 19. However, the time at which operations S2010 and S2020 are performed is not limited thereto.

[0258] In operation S2010, the electronic device 3000 may determine whether to activate at least one shifting logic of the multiplication logic, based on the identified datatypes and precisions. The electronic device 3000 may determine whether to activate the at least one shifting logic mapped to the identified datatypes and precisions. The electronic device 3000 may activate or deactivate the at least one shifting logic that shifts the outputs of the sub-multiplication logics according to an identification result.

[0259] In operation S2020, the electronic device 3000 may determine to activate the at least one shifting logic, and activate the at least one shifting logic that shifts at least one of the outputs of the sub-multiplication logics. In an embodiment of the disclosure, the activated shifting logic may perform left shifting on the outputs of the sub-multiplication logics by a predefined bit width. In an embodiment of the disclosure, the electronic device 3000 may obtain the output of the multiplication logic, based on the shifted or unshifted outputs of the sub-multiplication logics.

[0260] FIG. 21 is a flowchart for explaining the operation of a selective adder according to an embodiment of the disclosure. Matters of FIG. 21 that are the same as those described above with reference to FIGS. 1 to 20 will not be repeated herein.

[0261] In an embodiment of the disclosure, operations S2110 and S2120 may be performed by at least one of the electronic device 3000, the multiplier 3110, the hardware accelerator 3100, and the processor 3200 of FIG. 18. However, the disclosure is not limited thereto, and operations S2110 and S2120 may be performed by any electronic device. An operating method of the selective adder or a controlling method of the selective adder according to an embodiment of the disclosure is not limited to that shown in FIG. 21, and may omit at least one of the operations shown in FIG. 21 or may further include an operation not shown in FIG. 21.

[0262] In an embodiment of the disclosure, operations S2110 and S2120 may be performed by the electronic device 3000 between operations S1930 and S1940 of FIG. 19. However, the time at which operations S2110 and S2120 are performed is not limited thereto.

[0263] In operation S2110, the electronic device 3000 may determine whether to activate the selective adder of the multiplication logic, based on the identified datatypes and precisions. The electronic device 3000 may determine whether to activate the selective adder mapped to the identified datatypes and precisions. The electronic device 3000 may activate or deactivate the selective adder according to an identification result. For example, the electronic device 3000 may activate the selective adder when the identified datatypes and precisions are bfloat16. For example, the electronic device 3000 may deactivate the selective adder when the identified datatypes and precisions are FP8.

[0264] In operation S2120, the electronic device 3000 may determine to activate the selective adder, and activate the selective adder that sums the outputs of the sub-multiplication logics. In an embodiment of the disclosure, the activated selective adder may sum all the outputs of the sub-multiplication logics.

[0265] FIG. 22 is a flowchart for explaining the operation of addition logic according to an embodiment of the disclosure. Matters of FIG. 22 that are the same as those described above with reference to FIGS. 1 to 21 will not be repeated herein.

[0266] In an embodiment of the disclosure, operations S2210 and S2220 may be performed by at least one of the electronic device 3000, the multiplier 3110, the hardware accelerator 3100, and the processor 3200 of FIG. 18. However, the disclosure is not limited thereto, and operations S2210 and S2220 may be performed by any electronic device. An operating method of the addition logic or a controlling method of the addition logic according to an embodiment of the disclosure is not limited to that shown in FIG. 22, and may omit at least one of the operations shown in FIG. 22 or may further include an operation not shown in FIG. 22.

[0267] In an embodiment of the disclosure, operations S2210 and S2220 may be performed by the electronic device 3000 after operations S1940 of FIG. 19. However, the time at which operations S2210 and S2220 are performed is not limited thereto.

[0268] In operation S2210, the electronic device 3000 may determine whether to activate the addition logic, based on the identified datatypes and precisions. In an embodiment of the disclosure, the electronic device 3000 may determine whether to activate at least one sub-addition logic of the addition logic.

[0269] In operation S2220, the electronic device 3000 may determine to activate the addition logic, and may activate the addition logic that sums bits corresponding to an exponent of a second input value and bits corresponding to an exponent of a first input value, and subtracts bias values mapped to the identified datatypes and precisions from a result of the summation. The at least one sub-addition logic of the activated addition logic may perform addition between exponents by using two adders. One adder may sum bits corresponding to an exponent of the second input value and bits corresponding to an exponent of the first input value. Another adder may subtract bias values mapped to the identified datatypes and precisions from a summation result.

[0270] FIG. 23 is a flowchart for explaining the operation of XOR logic according to an embodiment of the disclosure. Matters of FIG. 23 that are the same as those described above with reference to FIGS. 1 to 22 will not be repeated herein.

[0271] In an embodiment of the disclosure, operations S2310 and S2320 may be performed by at least one of the electronic device 3000, the multiplier 3110, the hardware accelerator 3100, and the processor 3200 of FIG. 18. However, the disclosure is not limited thereto, and operations S2310 and S2320 may be performed by any electronic device. An operating method of the XOR logic or a controlling method of the XOR logic according to an embodiment of the disclosure is not limited to that shown in FIG. 23, and may omit at least one of the operations shown in FIG. 23 or may further include an operation not shown in FIG. 23.

[0272] In an embodiment of the disclosure, operations S2310 and S2320 may be performed by the electronic device 3000 after operations S2220 of FIG. 22. However, the time at which operations S2310 and S2320 are performed is not limited thereto.

[0273] In operation S2310, the electronic device 3000 may determine whether to activate the XOR logic, based on the identified datatypes and precisions. In an embodiment of the disclosure, the electronic device 3000 may determine whether to activate at least one sub-XOR logic of the XOR logic.

[0274] In operation S2320, the electronic device 3000 may determine to activate the XOR logic, and may perform an XOR operation between a bit corresponding to a sign of the first input value and a bit corresponding to a sign of the second input value.

[0275] FIG. 24 is a flowchart for explaining the operations of a normalizer and a rounder according to an embodiment of the disclosure. Matters of FIG. 24 that are the same as those described above with reference to FIGS. 1 to 23 will not be repeated herein.

[0276] In an embodiment of the disclosure, operations S2410 to S2430 may be performed by at least one of the electronic device 3000, the multiplier 3110, the hardware accelerator 3100, and the

processor **3200** of FIG. **18**. However, the disclosure is not limited thereto, and operations **S2410** to **S2430** may be performed by any electronic device. An operating method of the normalizer and the rounder or a controlling method of the normalizer and the rounder according to an embodiment of the disclosure is not limited to that shown in FIG. **24**, and may omit at least one of the operations shown in FIG. **24** or may further include an operation not shown in FIG. **24**.

[0277] In an embodiment of the disclosure, operations **S2410** to **S2430** may be performed by the electronic device **3000** after operations **S2320** of FIG. **23**. However, the time at which operations **S2410** to **S2430** are performed is not limited thereto.

[0278] In operation **S2410**, the electronic device **3000** may determine whether to activate the normalizer and the rounder, based on the identified datatypes and precisions. In an embodiment of the disclosure, the normalizer may include at least one sub-normalizer. The rounder may include at least one sub-rounder. The number of activated sub-normalizers may be equal to the number of activated sub-rounders.

[0279] In operation **S2420**, the electronic device **3000** may determine to activate the normalizer, and may activate the normalizer that normalizes at least one output of a multiplication logic and at least one output of an addition logic. In an embodiment of the disclosure, the number of sub-normalizers activated may vary depending on the identified datatypes and precisions.

[0280] In operation **S2430**, the electronic device **3000** may determine to activate the rounder, and may activate the rounder that performs rounding with a predefined bit width by using at least one output of the XOR logic and at least one output of the normalizer. In an embodiment of the disclosure, the number of sub-rounders activated may vary depending on the identified datatypes and precisions.

[0281] According to an embodiment of the disclosure, operations on various precisions and datatypes may be supported while maintaining the accuracy of training or inference of the DNN model.

[0282] According to an embodiment of the disclosure, an area-efficient multiplier that supports various precisions and datatypes may be provided. One multiplier may perform multiplication operations not only on floating-point types but also on integral types, and thus, a large number of multipliers may be integrated per unit area. In other words, there is no need to have different multipliers with respect to different precisions and datatypes.

[0283] According to an embodiment of the disclosure, a flexible and general-purpose MAC array with high hardware resource utilization with respect to various operations of the DNN model may be provided. The MAC array according to an embodiment of the disclosure is designed to have high resource utilization for both accumulated and non-accumulated operations, and one hardware accelerator is designed to process a plurality of DNN models in parallel.

[0284] For example, an application that provides augmented reality or virtual reality service needs to simultaneously perform tasks such as object detection and depth estimation, and thus, to provide the service of the application, the hardware accelerator may process the plurality of DNN models corresponding to the tasks in parallel.

[0285] In an embodiment of the disclosure, an electronic device may be provided. The electronic device may include a multiplier including a multiplication logic. The electronic device may include a memory including at least one instruction. The electronic device may include at least one processor configured to execute the at least one instruction. The at least one processor may obtain a first input value and a second input value. The at least one processor may identify datatypes and precisions of the first input value and the second input value. The at least one processor may, based on the identified datatypes and precisions, distribute bits of the first input value and bits of the second input value to sub-multiplication logics of the multiplication logic. The at least one processor may obtain at least one output of the multiplication logic, based on outputs of the sub-multiplication logics.

[0286] In an embodiment of the disclosure, the at least one processor may determine whether to

activate at least one shifting logic of the multiplication logic, based on the identified datatypes and precisions. The at least one processor may determine to activate the at least one shifting logic, and activate the at least one shifting logic configured to shift at least one of the outputs of the sub-multiplication logics.

[0287] In an embodiment of the disclosure, the at least one processor may determine whether to activate a selective adder of the multiplication logic, based on the identified datatypes and precisions. The at least one processor may determine to activate the selective adder, and activate the selective adder that sums the outputs of the sub-multiplication logics. The at least one processor may determine to deactivate the selective adder and deactivate the selective adder that sums the outputs of the sub-multiplication logics.

[0288] In an embodiment of the disclosure, when the identified datatypes are floating-point types, the bits of the first input value distributed to the sub-multiplication logics may be bits corresponding to a mantissa of the first input value.

[0289] In an embodiment of the disclosure, the bits of the second input value distributed to the sub-multiplication logics may be bits corresponding to a mantissa of the second input value.

[0290] In an embodiment of the disclosure, the multiplier may include an addition logic.

[0291] In an embodiment of the disclosure, the at least one processor may determine whether to activate the addition logic, based on the identified datatypes and precisions. The at least one processor may determine to activate the addition logic, and may activate the addition logic that sums bits corresponding to an exponent of the second input value and bits corresponding to an exponent of the first input value and subtracts bias values mapped to the identified datatypes and precisions from a result of the summation.

[0292] In an embodiment of the disclosure, the multiplier may include an XOR logic.

[0293] In an embodiment of the disclosure, the at least one processor may determine whether to activate the XOR logic, based on the identified datatypes and precisions. The at least one processor may determine to activate the XOR logic, and may activate the XOR logic configured to perform an XOR operation between a bit corresponding to a sign of the first input value and a bit corresponding to a sign of the second input value.

[0294] In an embodiment of the disclosure, the XOR logic may include at least one sub-XOR logic.

[0295] In an embodiment of the disclosure, the at least one sub-XOR logic may be arranged in the same number as a maximum number of at least one output of the multiplication logic.

[0296] In an embodiment of the disclosure, the multiplier may include a normalizer.

[0297] In an embodiment of the disclosure, the multiplier may include a rounder.

[0298] In an embodiment of the disclosure, the at least one processor may determine whether to activate the normalizer and the rounder, based on the identified datatypes and precisions. The at least one processor may determine to activate the normalizer, and may activate the normalizer that normalizes at least one output of the multiplication logic and at least one output of the addition logic. The at least one processor may determine to activate the rounder, and may activate the rounder that performs rounding with a predefined bit width by using at least one output of the XOR logic and at least one output of the normalizer.

[0299] In an embodiment of the disclosure, the at least one processor may, based on the identified datatypes being integral types, determine at least one output of the multiplication logic to be an output of the multiplier. The at least one processor may, based on the identified datatypes being floating-point types, determine the output of the normalizer as an output of the multiplier.

[0300] In an embodiment of the disclosure, an operating method of a multiplier may be provided. The method may include obtaining a first input value and a second input value. The method may include identifying datatypes and precisions of the first input value and the second input value. The method may include, based on the identified datatypes and precisions, distributing bits of the first input value and bits of the second input value to sub-multiplication logics of a multiplication logic included in the multiplier. The method may include obtaining at least one output of the

multiplication logic, based on outputs of the sub-multiplication logics.

[0301] In an embodiment of the disclosure, a multiplier may be provided. The multiplier may include a multiplication logic configured to, based on datatypes of a first input value and a second input value being integral types, perform multiplication between the first input value and the second input value, and, based on the datatypes being floating-point types, perform multiplication between a mantissa of the first input value and a mantissa of the second input value. The multiplier may include an addition logic configured to, based on the datatypes being floating-point types, sum an exponent of the first input value and an exponent of the second input value, and subtract bias values mapped to bit widths of the exponent of the first input value and the exponent of the second input value from a result of the summation. The multiplier may include an XOR logic configured to, based on the datatypes being floating-point types, perform an XOR operation between a sign of the first input value and a sign of the second input value. The multiplier may include a normalizer configured to, based on the datatypes being floating-point types, normalize an output of the multiplication logic and an output of the addition logic. The multiplier may include a rounder configured to, based on the datatypes being floating-point types, perform rounding with a predefined bit width by using an output of the normalizer and an output of the XOR logic. The multiplier may include a multiplexer configured to, based on the datatypes being integral types, output the output of the multiplication logic, and based on the datatypes being floating-point types, output an output of the rounder.

[0302] In an embodiment of the disclosure, a hardware accelerator may be provided. The hardware accelerator may include a MAC array. The MAC array may include a plurality of arrays. The MAC array may include a global multiplexer logic connecting or disconnecting at least two of the plurality of arrays, based on a first control signal. Each of the plurality of arrays may include a plurality of sub-arrays including a plurality of first sub-arrays and a second sub-array. Each of the plurality of arrays may include a local multiplexer logic connecting or disconnecting the plurality of sub-arrays, based on a second control signal.

[0303] In an embodiment of the disclosure, the MAC array may perform a 128×128 matrix operation. Each of the plurality of arrays may perform a 64×64 matrix operation.

[0304] In an embodiment of the disclosure, MAC operators may be arranged in each of the plurality of first sub-arrays in the form of 9×64 . MAC operators may be arranged in the second sub-array in the form of 1×64 .

[0305] In an embodiment of the disclosure, the number of arrays may be 4. The number of first sub-arrays may be 7.

[0306] In an embodiment of the disclosure, each of the MAC operators may include a multiplier that performs multiplication between an input value and a weight value. Each of the MAC operators may include an adder summing a partial sum value and an output of the multiplier. Each of the MAC operators may include a precision converter that converts precision of the output of the multiplier. Each of the MAC operators may include an adder tree summing the partial sum value and an output of the precision converter. Each of the MAC operators may include a first multiplexer that outputs an output of the adder or an output of the adder tree, based on a third control signal.

[0307] In an embodiment of the disclosure, based on the first control signal indicating a first operation mode of the MAC array, the global multiplexer logic may disconnect the plurality of arrays from each other.

[0308] In an embodiment of the disclosure, based on the first control signal indicating a second operation mode of the MAC array, the global multiplexer logic may connect at least two of the plurality of arrays to each other.

[0309] In an embodiment of the disclosure, the plurality of arrays may be grouped into at least one array group by the global multiplexer logic. Each of the at least one array group may perform an operation on different single tenants.

[0310] In an embodiment of the disclosure, based on the second control signal indicating a first operation mode of the plurality of arrays, the local multiplexer logic may disconnect the plurality of sub-arrays from each other.

[0311] In an embodiment of the disclosure, based on the second control signal indicating a second operation mode of the plurality of arrays, the local multiplexer logic may connect the plurality of sub-arrays to each other.

[0312] In an embodiment of the disclosure, the plurality of sub-arrays may generate a partial sum tensor by performing a convolution operation between a first tensor and a second tensor. Based on the second control signal indicating the first operation mode of the plurality of arrays, the plurality of arrays may accumulate the partial sum tensors respectively output by the plurality of sub-arrays. Based on the second control signal indicating the second operation mode of the plurality of arrays, the plurality of arrays may not accumulate the partial sum tensors respectively output by the plurality of sub-arrays.

[0313] In an embodiment of the disclosure, a MAC array may be provided. The MAC array may include four arrays, each including seven first sub-arrays, one second sub-array, and a local multiplexer logic. The MAC array may include a global multiplexer logic disposed between the arrays. Each of the first sub-arrays may include a plurality of MAC operators arranged in the form of 9×64 . The second sub-array may include a plurality of MAC operators arranged in the form of 1×64 . The local multiplexer logic may be disposed between the first sub-arrays and the second sub-array.

[0314] In an embodiment of the disclosure, the global multiplexer logic may include a plurality of first multiplexers connected between the plurality of MAC operators disposed in a last row, a last column, a first row, or a first column of the arrays.

[0315] In an embodiment of the disclosure, each of the plurality of first multiplexers may transfer an output (e.g., a partial sum or forwarded data) of one of the arrays to another array among the arrays, based on the first control signal corresponding to a first logic value. Each of the plurality of first multiplexers may not transfer the output (e.g., a partial sum or forwarded data) of one of the arrays to another array among the arrays, based on the first control signal corresponding to a second logic value.

[0316] In an embodiment of the disclosure, the local multiplexer logic may include a plurality of second multiplexers connected to the plurality of MAC operators arranged in a last row or a last column of the first sub-arrays.

[0317] In an embodiment of the disclosure, each of the plurality of second multiplexers may transfer an output of one of the first sub-arrays to another first sub-array or the second sub-array, based on the second control signal corresponding to the first logical value. Each of the plurality of second multiplexers may not transfer the output of one of the first sub-arrays to another first sub-array or the second sub-array, based on the second control signal corresponding to the second logical value.

[0318] In an embodiment of the disclosure, when the second control signal corresponds to the first logic value, each of the arrays may perform a 64×64 matrix operation.

[0319] In an embodiment of the disclosure, values of the weight tensor corresponding to an input channel may be respectively prefetched to the arrays in a column direction. Values of the weight tensor corresponding to an output channel may be respectively prefetched to the arrays in a row direction.

[0320] In an embodiment of the disclosure, when the second control signal corresponds to the second logic value, values of a weight filter of the weight tensor may be respectively prefetched to the first sub-arrays in the column direction. When the second control signal corresponds to the second logic value, the values of the weight tensor corresponding to the output channel may be respectively prefetched to the arrays in the row direction.

[0321] In an embodiment of the disclosure, when the second control signal corresponds to the

second logic value, the MAC operators of the second sub-array may be grouped into seven groups such that nine MAC operators are included in each group.

[0322] In an embodiment of the disclosure, the values of the weight tensor may be respectively prefetched to the seven groups.

[0323] A method according to an embodiment of the disclosure may be embodied as program commands executable by various computer means and may be recorded on a computer-readable recording medium. The computer-readable recording medium may include program commands, data files, data structures, and the like separately or in combinations. The program commands to be recorded on the computer-readable recording medium may be specially designed and configured for embodiments or may be well-known to and be usable by one of ordinary skill in the art of computer software. Examples of a computer-readable recording medium include a magnetic medium such as a hard disk, a floppy disk, or a magnetic tape, an optical medium such as a compact disk-read-only memory (CD-ROM) or a digital versatile disk (DVD), a magneto-optical medium such as a floptical disk, and a hardware device specially configured to store and execute program commands such as a ROM, a random-access memory (RAM), or a flash memory. Examples of the program commands are high-level language codes that can be executed by a computer by using an interpreter or the like as well as machine language codes made by a compiler.

[0324] An embodiment of the disclosure may also be implemented in the form of a recording medium including instructions executable by a computer, such as a program module executed by a computer. Computer-readable media may be any available media accessible by a computer and includes both volatile and nonvolatile media and removable and non-removable media. Further, the computer readable medium may include all computer storage and communication media. The computer storage medium includes all volatile/non-volatile and removable/non-removable media embodied by a certain method or technology for storing information such as computer readable instruction code, a data structure, a program module or other data. The communication medium typically includes the computer readable instruction code, the data structure, the program module, or other data of a modulated data signal, or other transmission mechanism, and includes any information transmission medium. An embodiment of the disclosure may be implemented as a computer program or a computer program product including instructions executable by a computer.

[0325] In an embodiment of the disclosure, a machine-readable storage medium may be provided in the form of a non-transitory storage medium. The 'non-transitory storage medium' is a tangible device and only means that it does not contain a signal (e.g., electromagnetic waves). This term does not distinguish a case in which data is stored semi-permanently in a storage medium from a case in which data is temporarily stored. For example, the non-transitory storage medium may include a buffer in which data is temporarily stored.

[0326] According to an embodiment of the disclosure, a method according to various disclosed embodiments may be provided by being included in a computer program product. The computer program product, which is a commodity, may be traded between sellers and buyers. Computer program products are distributed in the form of device-readable storage media (e.g., compact disc read only memory (CD-ROM)), or may be distributed (e.g., downloaded or uploaded) through an application store or between two user devices (e.g., smartphones) directly and online. In the case of online distribution, at least a portion of the computer program product (e.g., a downloadable app) may be stored at least temporarily in a device-readable storage medium, such as a memory of a manufacturer's server, a server of an application store, or a relay server, or may be temporarily generated.

[0327] While one or more embodiments have been described with reference to the figures, it will be understood by those of ordinary skill in the art that various changes in form and details may be made therein without departing from the spirit and scope as defined by the following claims.

Claims

1. A hardware accelerator comprising a multiply-accumulate (MAC) array, wherein the MAC array comprises: a plurality of arrays; and a global multiplexer logic configured to connect or disconnect at least two of the plurality of arrays, based on a first control signal, and each of the plurality of arrays comprises: a plurality of sub-arrays including a plurality of first sub-arrays and a second sub-array; and a local multiplexer logic configured to connect or disconnect the plurality of sub-arrays, based on a second control signal.
2. The hardware accelerator of claim 1, wherein the MAC array performs a 128×128 matrix operation, and each of the plurality of arrays performs a 64×64 matrix operation.
3. The hardware accelerator of claim 2, wherein MAC operators are arranged in each of the plurality of first sub-arrays in a form of 9×64 , and MAC operators are arranged in the second sub-array in a form of 1×64 .
4. The hardware accelerator of claim 3, wherein a number of the plurality of arrays is 4, and a number of the plurality of first sub-arrays is 7.
5. The hardware accelerator of claim 3, wherein each of the MAC operators comprises: a multiplier configured to perform multiplication between an input value and a weight value; an adder configured to sum a partial sum value and an output of the multiplier; a precision converter configured to convert precision of the output of the multiplier; an adder tree configured to sum the partial sum value and an output of the precision converter; and a first multiplexer configured to output an output of the adder or an output of the adder tree, based on a third control signal.
6. The hardware accelerator of claim 5, wherein the multiplier comprises: a multiplication logic configured to, based on datatypes of the input value and the weight value being integral types, perform multiplication between the input value and the weight value, and, based on the datatypes being floating-point types, perform multiplication between a mantissa of the input value and a mantissa of the weight value; an addition logic configured to, based on the datatypes being floating-point types, sum an exponent of the input value and an exponent of the weight value, and subtract bias values mapped to bit widths of the exponent of the input value and the exponent of the weight value from a result of the summation; an XOR logic configured to, based on the datatypes being floating-point types, perform an XOR operation between a sign of the input value and a sign of the weight value; a normalizer configured to, based on the datatypes being floating-point types, normalize an output of the multiplication logic and an output of the addition logic; a rounder configured to, based on the datatypes being floating-point types, perform rounding with a predefined bit width by using an output of the normalizer and an output of the XOR logic; and a second multiplexer configured to, based on the datatypes being integral types, output the output of the multiplication logic, and based on the datatypes being floating-point types, output an output of the rounder.
7. The hardware accelerator of claim 1, wherein based on the first control signal indicating a first operation mode of the MAC array, the global multiplexer logic disconnects the plurality of arrays from each other, and based on the first control signal indicating a second operation mode of the MAC array, the global multiplexer logic connects at least two of the plurality of arrays to each other.
8. The hardware accelerator of claim 1, wherein the plurality of arrays are grouped into at least one array group by the global multiplexer logic, and each of the at least one array group performs an operation on different single tenants.
9. The hardware accelerator of claim 1, wherein based on the second control signal indicating a first operation mode of the plurality of arrays, the local multiplexer logic disconnects the plurality of sub-arrays from each other, and based on the second control signal indicating a second operation mode of the plurality of arrays, the local multiplexer logic connects the plurality of sub-arrays to

each other.

10. The hardware accelerator of claim 9, wherein the plurality of sub-arrays generates a partial sum tensor by performing a convolution operation between a first tensor and a second tensor, based on the second control signal indicating the first operation mode of the plurality of arrays, the plurality of arrays accumulate the partial sum tensors respectively output by the plurality of sub-arrays, and based on the second control signal indicating the second operation mode of the plurality of arrays, the plurality of arrays do not accumulate the partial sum tensors respectively output by the plurality of sub-arrays.

11. A multiply-accumulate (MAC) array comprising: arrays, each including seven first sub-arrays, one second sub-array, and a local multiplexer logic; and a global multiplexer logic disposed between the arrays, wherein each of the first sub-arrays includes a plurality of MAC operators arranged in a form of 9×64 , the second sub-array includes a plurality of MAC operators arranged in a form of 1×64 , and the local multiplexer logic is disposed between the first sub-arrays and the second sub-array.

12. The MAC array of claim 11, wherein the global multiplexer logic includes a plurality of first multiplexers connected between the plurality of MAC operators disposed in a last row, a last column, a first row, or a first column of the arrays.

13. The MAC array of claim 12, wherein each of the plurality of first multiplexers transfers an output of one of the arrays to another array, based on the first control signal corresponding to a first logic value, and does not transfer the output of one of the arrays to the other array, based on the first control signal corresponding to a second logic value.

14. The MAC array of claim 11, wherein the local multiplexer logic includes a plurality of second multiplexers connected to the plurality of MAC operators arranged in a last row or a last column of the first sub-arrays.

15. The MAC array of claim 14, wherein each of the plurality of second multiplexers transfers an output of one of the first sub-arrays to another first sub-array or the second sub-array, based on the second control signal corresponding to the first logical value, and does not transfer the output of one of the first sub-arrays to another first sub-array or the second sub-array, based on the second control signal corresponding to the second logical value.

16. The MAC array of claim 15, wherein, based on the second control signal corresponding to the first logic value, each of the arrays performs a 64×64 matrix operation.

17. The MAC array of claim 16, wherein values of a weight tensor corresponding to an input channel are respectively prefetched to the arrays in a column direction, and values of the weight tensor corresponding to an output channel are respectively prefetched to the arrays in a row direction.

18. The MAC array of claim 15, wherein, based on the second control signal corresponding to the second logic value, values of a weight filter of a weight tensor are respectively prefetched to the first sub-arrays in a column direction, and values of the weight tensor corresponding to an output channel are respectively prefetched to the first sub-arrays in a row direction.

19. The MAC array of claim 18, wherein, based on the second control signal corresponding to the second logic value, MAC operators of the second sub-array are grouped into seven groups such that nine MAC operators are included in each group.

20. The MAC array of claim 19, wherein the values of the weight tensor are respectively prefetched to each of the seven groups.
